

Jindřich Třaskoš

Simulace mobilního robotu na trajektorii s překážkami

Bakalářská práce

Ostrava 2022/23



Jindřich Třaskoš

Simulace mobilního robotu sledujícího trajektorii s překážkami

Simulation of mobile robot
following a trajectory with obstacles

Bakalářská práce

Vedoucí práce:
Ing. Petr Oščádal

Ostrava 2022/23

Bibliografický záznam

Autor: Jindřich Třaskoš, TRA0086

Název práce: Simulace mobilního robotu na trajektorii

Studijní program: Strojírenství/robotika

Vedoucí práce: Ing. Petr Oščádal

Akademický rok: 2022/2023

Počet stran: 59

Klíčová slova: simulace, mobilní robot, sledování trajektorie, překážky, algoritmy

ANOTACE BAKALÁŘSKÉ PRÁCE

TŘASKOŠ, Jindřich. *Simulace mobilního robotu na trajektorii s překážkami*. Ostrava, 2023, Xs. Bakalářská práce. VŠB - Technická univerzita Ostrava, Fakulta strojní, Katedra robotiky. Vedoucí práce Ing. Petr Oščádal.

Bakalářská práce se zabývá vytvořením simulace mobilního robotu sledujícího trajektorii s překážkami. V první části se práce věnuje aktuálnímu použití těchto robotů, sensorům které tyto roboty používají a algoritmům, které slouží k řešení jednotlivých problémů. Dále práce obsahuje rozbor typů trajektorií a srovnání nejznámějších simulačních softwarů s popisem použitého softwaru CoppeliaSim.

Praktická část práce pojednává o vytvoření simulace mobilního robota na trajektorii s překážkami. Simulační model vychází z podrobného modelu navrhnutého v systému Creo Parametric a z reálných hardwarových komponent potřebných k jeho činnosti. Dále jde zde popsán postup převedení modelu do simulace a vytvoření řídicího skriptu obsahující algoritmy pro sledování či hledání trajektorie, objíždění překážek a controlleru, který volí jednotlivé algoritmy. Následně se praktická část věnuje generátoru trajektorie a překážek a zajištěním nekonečné smyčky dané simulace. Závěr práce se věnuje celkovému shrnutí bakalářské práce.

ANNOTATION OF BACHELOR THESIS

TRASKOS, Jindrich. *Simulation of mobile robot following a trajectory with obstacles*. Ostrava, 2023, X p. Bachelor's thesis. VSB – Technical University of Ostrava, Faculty of Mechanical Engineering, Department of Robotics. Thesis head Ing. Petr Oscadal.

The bachelor's thesis deals with creation of a simulation of mobile robot following a trajectory with obstacles. The first part deals with the current use of these robots, the sensors used by these robots and the algorithms used to solve individual problems. Furthermore, the research includes an analysis of the types of trajectories and comparison of the most well-known simulation software with a description of the selected software CoppeliaSim.

The practical part of this thesis deals with the creation of a simulation of a mobile robot on a trajectory with obstacles. The simulation model is based on a detailed model designed in the Creo Parametric and on the real hardware components required for its operation. Furthermore, the procedure for converting the model into simulation and the creation of a control script containing algorithms for trajectory tracking, obstacle avoidance and a controller that selects the individual algorithms is described. Subsequently, the practical part discusses the creation of a trajectory and obstacle generator and ensuring the infinite loop of the given simulation. Finally, the thesis concludes with bachelor thesis summary.

Obsah

Seznam použitých zkratk a odborných termínů.....	6
Seznam obrázků	6
Seznam tabulek.....	8
Úvod.....	9
Cíle práce.....	10
1 Rešerše.....	11
1.1Hobby roboti.....	11
1.2Soutěžní roboti	12
1.3Industriální roboti	12
1.4Srovnání komerčních systémů.....	13
1.5Simulační systémy.....	13
1.5.1 Nejpoužívanější simulační systémy pro robotiku	14
1.5.2 Srovnání.....	15
1.5.3 Simulátor CoppeliaSim	15
1.6Trajektorie	17
1.6.1 Spojitá křivka.....	17
1.6.2 Přerušená křivka	17
1.6.3 Křižovatky	18
1.7Senzory detekce trajektorie	18
1.7.1 Magnetická detekce	18
1.7.2 Detekce pomocí indukce	20
1.7.3 Vizuální detekce	21
1.7.4 Umístění senzorů	23
1.7.5 Srovnání typu senzorů detekce trajektorie	23
1.8Senzory detekce překážek.....	24
1.8.1 Kamera	24
1.8.2 LiDAR (Light Detection and Ranging)	24
1.8.3 Sonar (Sound Navigation and Ranging)	25
1.9Pohybové ústrojí.....	26
1.9.1 Konstrukce podvozku	26
1.9.2 Pohon	26
1.10 Simulace v CoppeliaSim.....	27
1.10.1 Shape.....	27
1.10.2 Joint.....	28
1.10.3 Způsob detekce trajektorie v simulačním prostředí	28
1.10.4 Způsoby detekce překážek v simulačním prostředí	28

1.11	Algoritmus pro sledování trajektorie	29
1.12	Algoritmy pro vyhýbání se překážkám	30
1.12.1	Bug algoritmy	30
1.12.2	Bubble algoritmus	30
2	Požadavkový list.....	32
3	Praktická část	33
3.1	Model robota.....	33
3.1.1	Nosná konstrukce.....	34
3.1.2	Lokomoční subsystém	34
3.1.3	Senzorický subsystém.....	37
3.1.4	Mikrokontroler.....	40
3.1.5	Baterie	40
3.1.6	Spojovací komponenty.....	41
3.2	Simulace	41
3.2.1	Převedení do simulace	41
3.2.2	Řídící script.....	43
3.2.3	Generátor trajektorie a překážek.....	45
3.2.4	Kamera	46
4	Realizace	47
5	Závěr	48
6	Poděkování	49
7	Přílohy.....	50
7.1	Příloha A	50
7.2	Příloha B	51
7.3	Příloha C	51
7.4	Příloha D.....	52
7.5	Příloha E	53
7.6	Ostatní přílohy	54
8	Bibliografie.....	55

Seznam použitých zkratk a odborných termínů

SLAM	(anglická zkratka pro Simultaneous Localization and Mapping)
LED	(anglická zkratka pro Light emitting diode) dioda vyzařující světlo
Open-source	(„otevřený software“) software, který lze zdarma šířit a modifikovat.
ROS	(anglická zkratka pro Robot Operating System) software určený k vývoji robotických systémů
GUI	(anglická zkratka pro Graphical User Interface) grafické uživatelské rozhraní
Callback funkce	Jedná se o software funkci, která se volá při nastavené události
IR LED	(Infra) Dioda vyzařující infračervené záření
CoppeliaSim	Simulační software pro robotické systémy
Line follower	Robot sledující čáru
Creo	3d CAD software
CAD	(anglická zkratka pro Computer Aided Design) počítačem podporované navrhování
FOV	(anglická zkratka pro Field Of View) rozsah vidění
Caster wheel	Kuličkové kolo
STL	(anglická zkratka pro Standard Triangle Language) datový formát tvaru objektu
FDM	(anglická zkratka pro Fused Deposition modeling) typ procesu 3d tisku

Seznam obrázků

Obr. 0-1 Příklad vygenerované mapy pomocí metody SLAM.....	9
Obr. 0-2 Ukázka simulace robota	10
Obr. 1-1 Algoritmus robota „Emily“	11
Obr. 1-2 Hobby robot Elegoo	12
Obr. 1-3 Soutěžní robot LineCraft [3]	12
Obr. 1-4 Průmyslový robot ve smyčce [5]	13
Obr. 1-5 Simulační software Gazebo	14
Obr. 1-6 Hallův jev	19
Obr. 1-7 Magnetický senzor MGS1600C [9]	19
Obr. 1-8 Princip detekce trajektorie	20
Obr. 1-9 Nepřímé zapojení photo-coupleru	22
Obr. 1-10 Obrazový snímač [11]	22
Obr. 1-11 Stereo kamerový systém	24
Obr. 1-12 LiDAR v simulačním softwaru Gazebo	25
Obr. 1-13 Ultrazvukový senzor HC-SR04	26
Obr. 1-14 H-bridge driver pro DC motor	26
Obr. 1-15 Jednoduchý model v CoppeliaSim.....	27
Obr. 1-16 Hierarchie mobilního robota	27
Obr. 1-17 Vision senzory [12]	28
Obr. 1-18 Proximity senzory [13].....	28

Obr. 1-19 Regulační obvod pro sledování trajektorie	29
Obr. 1-20 Bug1 algoritmus [16]	30
Obr. 1-21 Bug2 algoritmus [17]	30
Obr. 1-22 Bubble algoritmus 1 [18]	31
Obr. 1-23 Bubble algoritmus 2 [18]	31
Obr. 3-1 Model robota	33
Obr. 3-2 Nosná konstrukce.....	34
Obr. 3-3 Lokomoční subsystém	35
Obr. 3-4 Lokomoční subsystém pohledy.....	35
Obr. 3-5 Motor [20]	36
Obr. 3-6 Driver DRV8835 [21].....	36
Obr. 3-7 Kola 1S2622 [22]	37
Obr. 3-8 Pololu Caster wheel [23].....	37
Obr. 3-9 Senzorický subsystém.....	37
Obr. 3-10 FOV Senzoru vzdálenosti	38
Obr. 3-11 GP2Y0A21YK0F [25].....	38
Obr. 3-12 FOV senzoru trajektorie.....	39
Obr. 3-13 QTR-MD-08A [26]	39
Obr. 3-14 MUX CD74HC4067 [27]	39
Obr. 3-15 Arduino UNO [28].....	40
Obr. 3-16 Arduino UNO-pins [28]	40
Obr. 3-17 Baterie G3 [29]	40
Obr. 3-18 Visuální model.....	41
Obr. 3-19 Dynamický model.....	41
Obr. 3-20 Vision senzor	42
Obr. 3-21 FOV senzoru GP2Y0A21YK0F	42
Obr. 3-22 výpočet úhlu	42
Obr. 3-23 FOV proximity senzoru	42
Obr. 3-24 Hierarchie robotu	43
Obr. 3-25 Graf odchylky	43
Obr. 3-26 Ukázka simulace.....	44
Obr. 3-27 Řetězec bodů	45
Obr. 3-28 Princip doplnění bodů	46
Obr. 3-29 Detekce kolize	46
Obr. 4-1 Realizace robota.....	47

Seznam tabulek

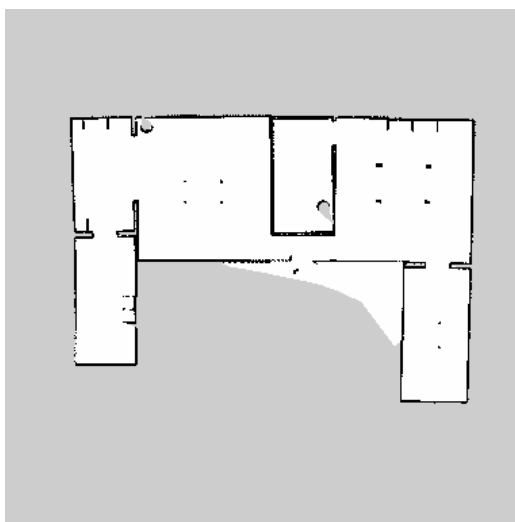
Tab. 1-1 Srovnání parametrů komerčních systémů dle důležitosti	13
Tab. 1-2 Srovnání simulačních softwarů 1 – umožňuje; 0 – neumožňuje.....	15
Tab. 1-3 Hlavní objekty scény v CoppeliaSim	16
Tab. 1-4 Spojité křivky.....	17
Tab. 1-5 Přerušené křivky.....	17
Tab. 1-6 Křižovatky.....	18
Tab. 1-7 Srovnání typu senzorů detekce trajektorie.....	23
Tab. 1-8 Hlavní parametry motorů	26
Tab. 2-1 Hodnotové požadavky	32
Tab. 2-2 Softwarové funkce	33
Tab. 3-1 Lokom. Subsystém komponenty	35
Tab. 3-2 Lokom. Subsystém parametry	35
Tab. 3-3 Parametry motoru [20].....	36
Tab. 3-4 Parametry DRV8835 [21]	36
Tab. 3-5 Senzor. subsystém komponenty	37
Tab. 3-6 Parametry GP2Y0A21YK0F [25].....	38
Tab. 3-7 Parametry senzoru trajektorie	39
Tab. 3-8 Parametry QTR-MD-08A [26]	39
Tab. 3-9 Parametry MUX CD74HC4067 [27]	39
Tab. 3-10 Parametry Baterie G3 [29]	40
Tab. 3-11 Parametry simulačního modelu	42

Úvod

Roboti se dají dle pohybu rozdělit na statické a mobilní. Statiční roboti nedokážou globálně měnit polohu základny. Jeden z příkladů tohoto typu robotu můžou být průmyslové roboty. Mobilní roboti můžou měnit globální polohu. Pokud chceme dostatečně využít schopnost mobility, je nutné vytvořit systém, který umožní kontrolovaný pohyb robota s přesností dle aplikace.

K tomu aby byl robot schopný se efektivně pohybovat v prostoru, je nutné aby robot dokázal detekovat, kde se nachází a zároveň jak se dostat na jinou pozici.

Jeden ze způsobů vytvoření kontrolovaného pohybu může být použití algoritmu SLAM a algoritmu pro navigaci. Robot používající SLAM je schopný pomocí senzorů vytvořit mapu prostředí (OBR. 0-1) a zároveň určit, kde se na mapě nachází. Následně pomocí navigace dle vytvořené mapy je schopen dostat se na pozici, kterou uživatel požaduje. Obecně je pro tvorbu tohoto systému nutné použití komplexního softwaru či dražších senzorů. Tento systém se například používá pro automatické vysavače, nebo sekačky na trávu.

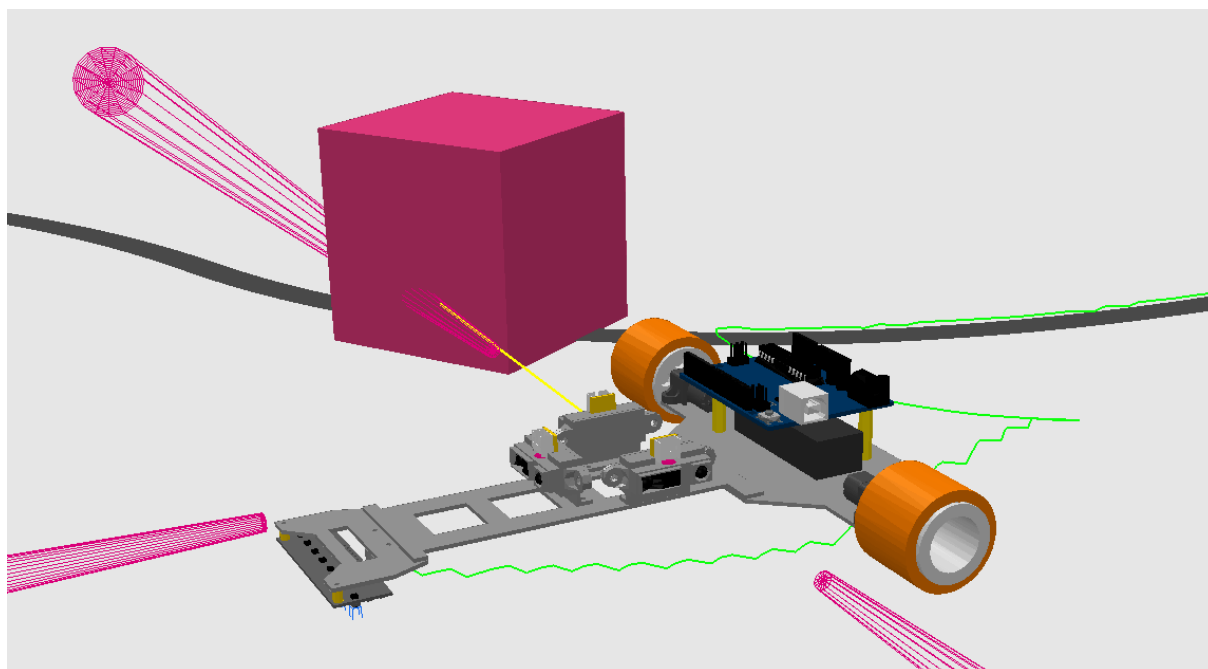


Obr. 0-1 Příklad vygenerované mapy pomocí metody SLAM

Pro některé aplikace není nutné, aby byl robot schopný určit přesnou polohu v prostoru. Pokud se robot bude pohybovat po statické trase, stačí tuto trasu vyznačit a zajistit aby se robot po ní dokázal pohybovat. Tento způsob kontrolovaného pohybu mobilního robota se nazývá sledování trajektorie, či sledování čáry. Trajektorie není obvykle synonymum s pojmem čára. V této práci se ale jedná o synonymum.

Sledování trajektorie umožňuje přesnou kontrolu pohybu robota. Díky vyznačení trasy je u robota možné „nastavit“ kudy se robot bude pohybovat. Výhoda tohoto způsobu pohybu je v úpravě trasy dle potřeby. Zároveň ale není nutné přeprogramovat software, pro jinou trasu. Hlavní problematiky této metody pohybu robota, jsou způsoby vyznačení a detekce trajektorie, pohyb po

dané trajektorii a logika při speciálních situacích. Tyto speciální situace můžou být např.: detekována křižovatka, překážka před robotem, či přerušená trajektorie.



Obr. 0-2 Ukázka simulace robota

Roboti jsou v dnešní době řešeni dvěma hlavními způsoby, a to reálně a simulací. Často tyto dva postupy jdou ruku v ruce. Reálné řešení neboli postavení prototypu poskytuje často finanční problém. Pokud bychom postavili prototyp a zjistili nedostatek v řešení, museli bychom postavit prototyp nový. Oproti tomu simulace poskytuje neomezené testování robota a z toho důvodu je simulace zařazena jako jeden z prvotních kroků ve vývoji finančně náročných robotů.

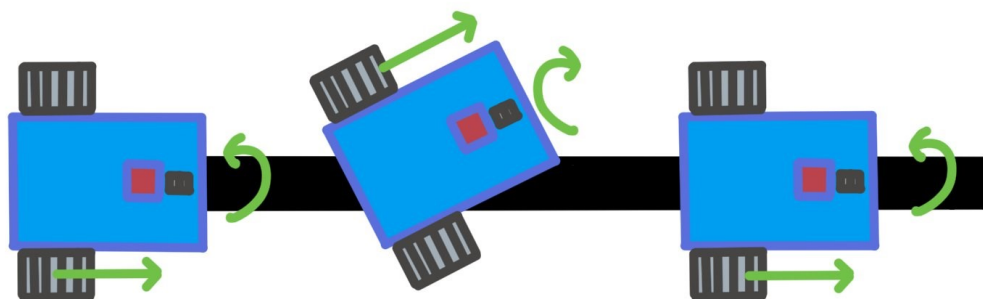
Cíle práce

V první části se budu zabývat metodou sledování trajektorie a řešení překážek. Zde spadají komerční systémy využívající sledování trajektorie, typy trajektorií, senzory pro detekci trajektorie a překážek a algoritmy pro sledování trajektorie a vyhýbání se překážkám. Dále se v této části budu zabývat známými simulačními systémy pro tento typ aplikace.

Praktická část práce se bude věnovat vytvořením simulace robota sledujícího trajektorii v softwaru CoppeliaSim. Simulační model robota bude vycházet z podrobného modelu a z reálných komponent. Robot bude mít implementovaný řídicí skript, který zajistí sledování dynamické trajektorie a objíždění překážek. V simulaci bude implementován skript pro generaci trajektorie a překážek.

1 Rešerše

Roboti, kteří sledují trajektorii jsou v dnešní době velmi známí. Jeden z pravděpodobně prvních designů, který byl zveřejněný se objevil v roce 1962. [1] Tento robot „Emily“ byl schopný sledovat bílou čáru. Vstupní pár senzorů byl seléniový snímač světla a signální LED. [2] Konstrukce robotu měla 2 kola, na které byly napojeny DC motory a jedno volné kolo pro udržení stability. Robot střídavě zapínal pravý/levý motor podle toho jestli detekoval trajektorii pod senzorem. Na tomto principu funguje algoritmus pro sledování trajektorie. Robot detekuje vzdálenostní odchylku středu robota od trajektorie a kompenzuje odchylku spuštěním motorů ve správném směru. (OBR. 1-1)



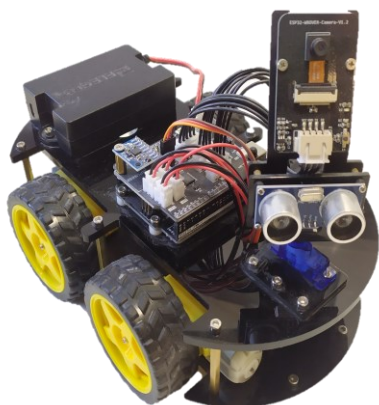
Obr. 1-1 Algoritmus robota „Emily“

Robot „Emily“ Měl pouze jeden pár čtecích senzorů. Z toho důvodu se robot pohyboval po trajektorii neplynulým pohybem a musel velmi rychle střídat zapínání motorů.

V dnešní době se dají rozdělit komerční roboti na sledování trajektorie na 3 hlavní skupiny.

1.1 Hobby roboti

Hobby roboti slouží k otestování dovedností programování a elektroniky, popř. jako ukázka robotiky. (OBR. 1-2) Tito jednodušší roboti se dají koupit ve vývojových sadách, či jako celistvý robot pro děti. Často jsou tyto sady postaveny na běžných hobby mikrokontrolerech jako jsou Arduino, nebo RaspberryPi. Tito roboti jsou velmi běžní a cenově dostupní. Jejich hlavní funkce je detekce jednoduché trajektorie, bez velkých změn. Pokud by došlo například k velmi ostré zatáčce na trajektorii, nebo pokud by motory byly spuštěny na maximální rychlost, často nebudou stíhat čáru detekovat a vyjedou z dráhy.

*Obr. 1-2 Hobby robot Elegoo**Obr. 1-3 Soutěžní robot LineCraft [3]*

1.2 Soutěžní roboti

Druhý pokročilejší typ jsou soutěžní roboti. Jedny z nejčastějších soutěží v robotice se věnují tématu sledování čáry. Existují různé typy soutěží. Ve všech soutěžích je primární způsob měření kvality robotu čas, za který projede danou trať. Dále záleží jestli se soutěž zaměřuje jenom na rychlost, nebo i na problémy, které se na dráze můžou vyskytnout (např. přerušení tratě). Při soutěžích, které se zaměřují primárně na rychlost, se roboti v konstrukci zásadně neliší. Roboti jsou konstruováni na základě poměru hmotnosti a výkonu. (OBR. 1-3) Hlavní část váhy je v zadní části robota, v přední části jsou pouze senzory. Celková konstrukce má nízký profil a snaží se docílit co nejnižší postaveného těžiště. Díky této konstrukci, jsou roboti schopni velkých rychlostí a to i při velmi ostrých zatáčkách.

Některé tyto rychlostí soutěže umožňují i prvotní projetí dráhy robotem. Díky tomu může robot v příštím kole predikovat, kde se nachází rovné části trajektorie a může na nich zvýšit rychlost. S kvalitou daného hardwaru se jejich cena stupňuje. Software daného robota, je upravován přesně na míru jejich konstrukce.

1.3 Industriální roboti

V průmyslu se taktéž objevují roboti sledující čáru. Hlavní účel těchto robotů bývá v přemísťování nákladu po určitých trasách. Tito roboti jsou robustnější z důvodu pravidelného převozu těžkého nákladu. Z důvodu bezpečnosti obsahují mechanismus pro detekci překážky.

Přímé využití těchto robotů můžeme najít např. u automatizované výroby aut Tesla. [4] Robot zde slouží k převozu aut mezi jednotlivými výrobními procesy. Jiný příklad může být robot, který slouží jako nákladní vozík. (OBR. 1-4) Tento robot jezdí ve smyčce v okolí výrobních míst. Jakmile uživatel oznámí,

že na pracovišti je náklad, robot při projíždění kolem zastaví. Po následném naložení odveze náklad nejkratší možnou cestou na výdej. Někteří industriální roboti pracují s nákladními vozíky. Tito roboti najedou pod vozík, zamknou vozík ke konstrukci, a pohybují se dále po trajektorii i s připevněným vozíkem.



Obr. 1-4 Průmyslový robot ve smyčce [5]

1.4 Srovnání komerčních systémů

Parametry	Hobby roboti	Soutěžní roboti	Industriální
Cena	do 3000 Kč	3000 Kč a výše	>> vyšší
Rychlost pohybu	0	2	1
Přesnost detekce trajektorie	0	2	2
Rychlost detekce trajektorie	0	2	2
Hmotnost robota	0	2	1
Únostnost robota	0	0	2
Adaptabilita algoritmu	0	2	2
Jednoduchost ovládání	1	0	2
Legenda	Označení		
nedůležité	0		
důležité	1		
vysoce důležité	2		

Tab. 1-1 Srovnání parametrů komerčních systémů dle důležitosti

1.5 Simulační systémy

Simulační systémy jsou v robotice velmi rozšířené. Veškeré projekty finančně nákladných robotů používají při návrhu simulační systémy. Vytvoření prvotní simulace robota a jeho návrhu, je několikanásobně levnější, než vytvářet návrh a provádět testování chování robota okamžitě v reálném prostředí. Hardware a vývoj robota je velmi nákladný, simulace nám umožňuje efektivní testování koncepčních návrhů bez závislosti na hardwaru. Další výhody simulace jsou v možnosti vytvoření prostředí bez nutnosti realizace. V simulaci lze jednoduše změnit fyziku daného světa, lze upravit např. gravitaci a čas.

Výběr simulačního systému závisí primárně na typu robota, nebo robotů a jejich interakci s prostředím. Simulační systém by měl poskytovat „physics engine“, který umožňuje vytvoření předpokládaného chování reálného světa, detekci kolizí a tření, GUI, knihovny pro propojení simulace s programovacími jazyky a možnost vytvoření modelu kinematiky, pohonů a senzorů robota.

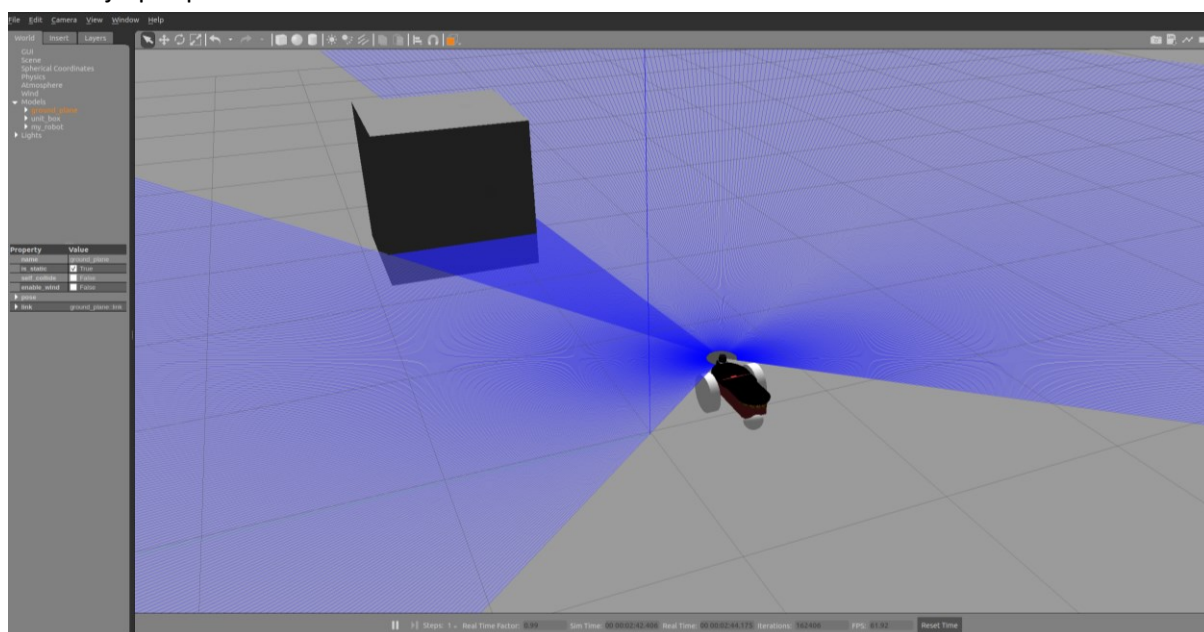
Simulační softwary lze rozlišit podle struktury robotů a prostředí ve kterém budou provádět činnost. Robot detekující trajektorii je mobilní robot, který se pohybuje po zemi. Existují vytvořené postupy pro výběr simulátoru dle aplikace robota. [6]

1.5.1 Nejpoužívanější simulační systémy pro robotiku

Z počtů citací pro zahrnuté simulátory mezi lety 2016 a 2020 a vyfiltrované slovem „robotics“ byly výsledovány nejcitovanější simulátory pro robotiku v tomto období. Nejvíce citované simulátory byly v těchto letech Gazebo, MuJoCo a CoppeliaSim. [7] V případě této práce se jedná o vytvoření simulace mobilního robota a daný simulační software by měl splňovat požadavky pro tento typ simulace. Hlavní požadavky pro simulátory mobilní robotiky jsou: umožnění pohybu robota, navigace v prostředí, řízení, vnímání světa pomocí senzorů, možnost kolaborace mezi více roboty, popř. možnost manipulace s objekty.

1.5.1.1 Gazebo

Gazebo je open-source simulační software vytvořený pro zjednodušení vývoje vysoce-výkonných aplikací. (OBR. 1-5) Gazebo byl vytvořen v roce 2002 na „University of Southern California“. Hlavní výhody tohoto systému jsou přímá integrace se systémem ROS, open-source a široký rozsah typů mobilních systémů jako jsou pásové, kolové, nebo kráčející systémy. Hlavní nedostatky Gazeba tkví v uživatelské nepřívětivosti. V Gazebu nelze přímo v simulaci upravovat robota a je nutné použít Gazebo soubor typu SDF (XML-formát), k jeho definici. Tento problém zpomaluje celkový vývoj a je náročnější pro prvotní naučení.



Obr. 1-5 Simulační software Gazebo

1.5.1.2 CoppeliaSim

CoppeliaSim / V-REP je simulátor vytvořený v Curychu ve Švýcarsku. Je to komerční systém, který lze získat zdarma jako studentskou verzi. CoppeliaSim podporuje systémy Windows, Linux a Mac. Tento systém je velmi podobný simulačnímu softwaru Gazebo, ale zároveň nemá jeho nevýhody. V CoppeliaSim je možné roboty upravovat přímo v simulačním prostředí a je snazší upravovat simulační prostředí pomocí GUI. CoppeliaSim obsahuje plugin, který umožňuje kooperaci se systémem ROS, pomocí defaultního jazyku Lua.

1.5.1.3 MuJoCo

MuJoCo (Multi-Joint dynamics with Contact) je fyzikální engine s pomocnými knihovnami. Mezi tyto knihovny patří vizualizace přes GUI, renderer OpenGL, integrace s pythonem, nebo použití Unity. Engine je určený pro výzkum a vývoj v robotice, biomechanice, strojového učení a dalších přesných simulací struktur, které interagují s prostředím. Tento engine je open-source, dostupný na githubu deepmind. Oproti ostatním zmíněným simulátorům neposkytuje podporu knihoven pro robotické systémy jako řešení inverzní-kinematiky a path planning. Samotnou implementaci je nutné navrhnout bez pomocných knihoven. Z tohoto důvodu není tento software zmíněný ve srovnání simulačních softwarů.

1.5.1.4 Další simulační softwary pro mobilní robotiku

Existují i další simulační softwary, které ale nejsou v tak velké míře využívány jako CoppeliaSim, nebo Gazebo. Jeden ze známých open-source simulačních softwarů je Webots. Další možnost simulátoru je např. PyBullet, což je open-source Python knihovna, která umožňuje vývoj robotických systémů a umožňuje další kooperaci s jinými Python knihovnami, např. knihovnou pro vývoj umělé inteligence.

1.5.2 Srovnání

Simulator	GPS	Pásy	Kola	Nohy	Omni-kola	Výškový terén	Navigace	ROS podpora	RGBD	LiDAR
Gazebo	1	1	1	1	1	1	1	1	1	1
CoppeliaSim	1	1	1	1	1	1	1	1	1	1
Webots	1	1	1	1	1	1	1	1	1	1
PyBullet	0	0	1	1	0	1	1	0	1	1

Tab. 1-2 Srovnání simulačních softwarů 1 – umožňuje; 0 – neumožňuje

1.5.3 Simulátor CoppeliaSim

Pro simulaci mobilního robota na trajektorii vybírám simulační software CoppeliaSim, z důvodu jednodušších úprav robota přímo v simulačním prostředí.

CoppeliaSim je simulátor, který je vytvořen na systému „objekt-skript“. Tento systémový styl je v dnešní době často používaný u softwarů, které slouží k vytváření simulací, počítačových her či práci s virtuálním prostředím. Používání těchto systémů je v zásadě podobné. Jde o použití datové struktury

(objekt) a vytvoření programu, který s tímto objektem spolupracuje. Tyto programy se nazývají skripty a jsou spojené s jedním anebo více objekty. Díky struktuře objekt-skript je možné vytvořit hierarchii objektů a skriptů s těmito objekty spojenými. [8]

1.5.3.1 Objekty

Objekt na nejvyšším stupni hierarchie, se kterým se v simulátoru CoppeliaSim pracuje, se nazývá scéna. Scéna obsahuje další subobjekty, které se nazývají „objekty scény“. Scéna může v CoppeliaSim obsahovat neomezený počet skriptů a objektů.

Objekty scény jsou hlavní objekty, ze kterých scénu skládáme. CoppeliaSim obsahuje 12 základních typů objektů scény. (TAB. 1-3) Tyto objekty mohou mít různé vlastnosti, které závisí na typu daného objektu.

Hlavní objekty scény v CoppeliaSim		
Název v angličtině	Název v češtině	Popis
Shape	Tvar	Objekt umožňující definici sítě pro určitý tvar.
Joint	Kloub	Umožňuje definici pohybu mezi objekty
Proximity sensor	Senzor detekce objektů	Umožňuje detekci objektů.
Vision sensor	Senzor vizuální detekce	Senzor ve stylu kamery.
Force/torque sensor	Silový/momentový senzor	Senzor umožňující zjištění sil a momentů, mezi objekty.
Octree	Octree	Datová struktura pro záznam 3d objektů.
Camera	Kamera	Umožňuje sledování scény.
Light	Světlo	Iluminační objekt vydávající světlo.
Dummy	Figurína	Bod s orientací v prostoru, který umožňuje více použití.
Point cloud	Point cloud	Typ datové struktury Octree.
Graph	Graf	Používá se k vizualizaci a záznamu dat.
Path	Cesta	Sdružení bodů s orientací v prostoru.

Tab. 1-3 Hlavní objekty scény v CoppeliaSim

Sdružení objektů scény se nazývá model. Model obsahuje objekty scény v definované hierarchii. Robot může obsahovat například dva objekty Shape, mezi kterými je objekt Joint. Dále může obsahovat např. senzorové objekty jako je Proximity sensor. Model je souborový typ „.ttm“, který lze uložit a následně načíst do scény.

1.5.3.2 Skripty

Programování simulace, scény a samotných objektů je z největší části prováděno pomocí skriptů. CoppeliaSim obsahuje integrovaný jazyk pro psaní skriptů „Lua“ a další externí jazyky např. Python a C++. Skripty se dají rozdělit na vestavěné (embedded) a externí. Vestavěné skripty jsou asociované se scénou nebo modelem a při načtení scény, nebo vložení modelu do simulace jsou skripty s nimi spojené. Externí skripty nejsou závislé na scéně nebo objektu.

Nejpoužívanější embedded skripty je hlavní skript, který je spojený se scénou a child skript, který je spojen s objekty. Child skript je simulační skript, který je spojený, nebo asociovaný s objekty scény.

Díky hlavnímu skriptu je možné danou scénu v simulaci spustit. Tento skript obsahuje callback funkce, které načítají a aktualizují objekty scény a child skripty. Načítání a aktualizace probíhá v pořadí daném hierarchií.

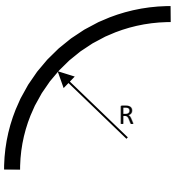
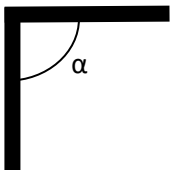
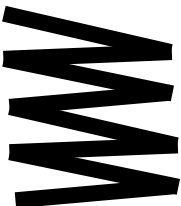
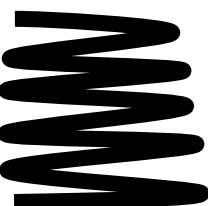
1.6 Trajektorie

Trajektorie je dráha, po které se robot pohybuje. Protože je to klíčový prvek systému, musí být předem dané jakým způsobem bude trajektorie zaznačena. Podle tvaru trajektorie a jejího značení, je následně vybrán způsob detekce a jaké senzory jsou pro detekci nutné.

Podle tvaru trajektorie se trajektorie dají rozdělit na spojitou křivku, přerušenou křivku a křižovatky. Nejdůležitější je spojitá křivka, protože z tohoto tvaru ostatní trajektorie vychází.

1.6.1 Spojitá křivka

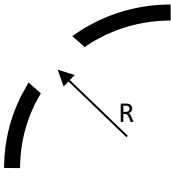
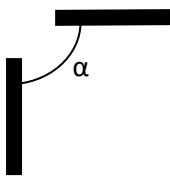
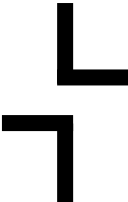
Spojitá křivka může mít jakýkoliv tvar. Pro jednoduchost a normalizaci se dají definovat tyto nejčastější případy:

				
Rovná čára	Zatáčka zaoblená dle R	Ostrá zatáčka dle úhlu α	Série ostrých zatáček	Série zaoblených zatáček

Tab. 1-4 Spojité křivky

1.6.2 Přerušená křivka

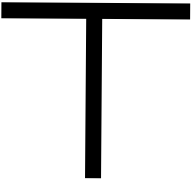
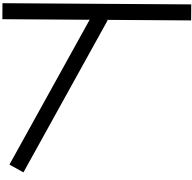
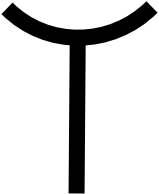
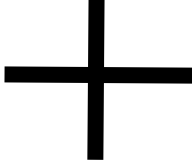
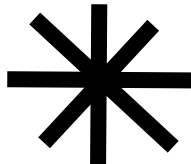
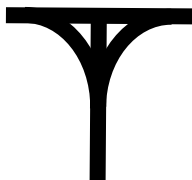
Přerušená křivka může být jakákoliv spojitá křivka s přerušením v jakémkoliv místě. Lze definovat například tyto obecné případy.

				
Rovná čára s přerušením	Zatáčka zaoblená dle R s přerušením	Ostrá zatáčka dle úhlu α s přerušením	Dvojité L	Dvojité J

Tab. 1-5 Přerušené křivky

1.6.3 Křižovatky

Spojení více spojitých křivek definuje křižovatku. Existuje řada možností jaké typy křižovatek definovat, lze postupovat například tímto způsobem:

					
T křižovatka	T pod úhlem	T oblá	+	Hvězda	T s oblouky

Tab. 1-6 Křižovatky

1.7 Senzory detekce trajektorie

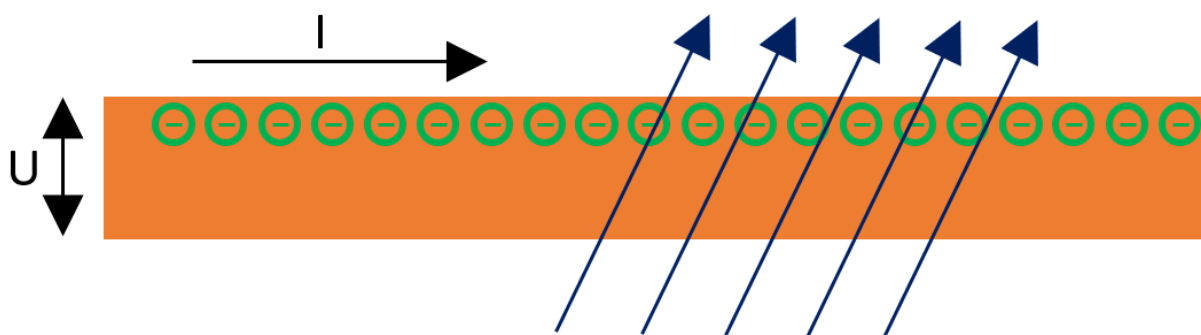
Senzory pro detekci trajektorie musí být schopné nejenom detekovat trajektorii, ale i komunikovat s řídicím systémem. Senzory pro detekci trajektorie pracují nejčastěji na 3 principech, a to princip magnetické detekce, detekce za použití indukce a vizuální detekce. Vizuální detekce lze rozdělit na detekci za pomoci kamery a detekci za pomoci IR senzoru.

Senzory pro detekci lze rozdělit na digitální a analogové. Analogové senzory poskytují přímý výstup informace. Informační výstup těchto analogových senzorů je nejčastěji v rozsahu napětí 0-10 V, nebo v proudovém rozsahu 4-20 mA. Digitální senzory interně převádí analogový signál na digitální. Digitální signál je výstupní hodnota zakódována jako binární číslo.

Analogové signály dokáže ovlivnit elektromagnetické pole. Digitální signály jsou vůči elektrickému rušení imunní. Digitální senzory zjednodušují přenos informace do řídicího systému, protože řídicí systém nepotřebuje digitalizovat analogový vstup. Zároveň ale při digitalizaci dochází ke ztrátě informace, proto je mít nutné na paměti jaký typ senzoru se pro danou aplikaci použije.

1.7.1 Magnetická detekce

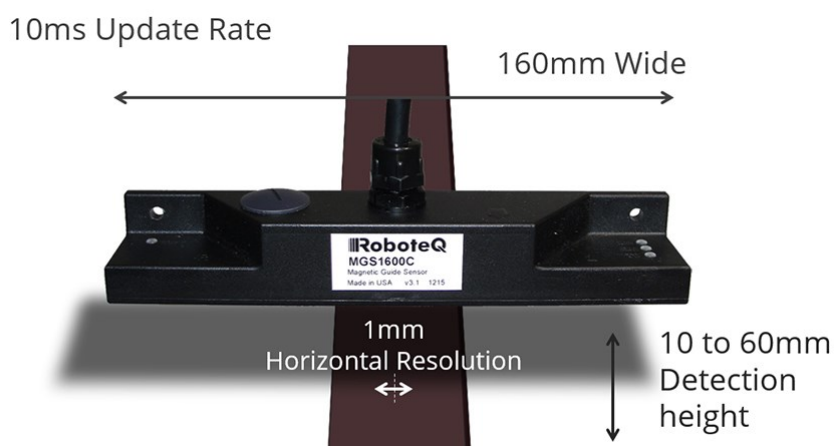
Magnetická detekce zahrnuje použití senzoru pro detekci magnetického pole a zaznačení trajektorie pomocí magnetického pásu. Nejčastěji se používá Hallův senzor, či Hallova sonda. Hallovy senzory využívají fyzikálního principu tzv. Hallova jevu. (OBR. 1-6)



Obr. 1-6 Hallův jev

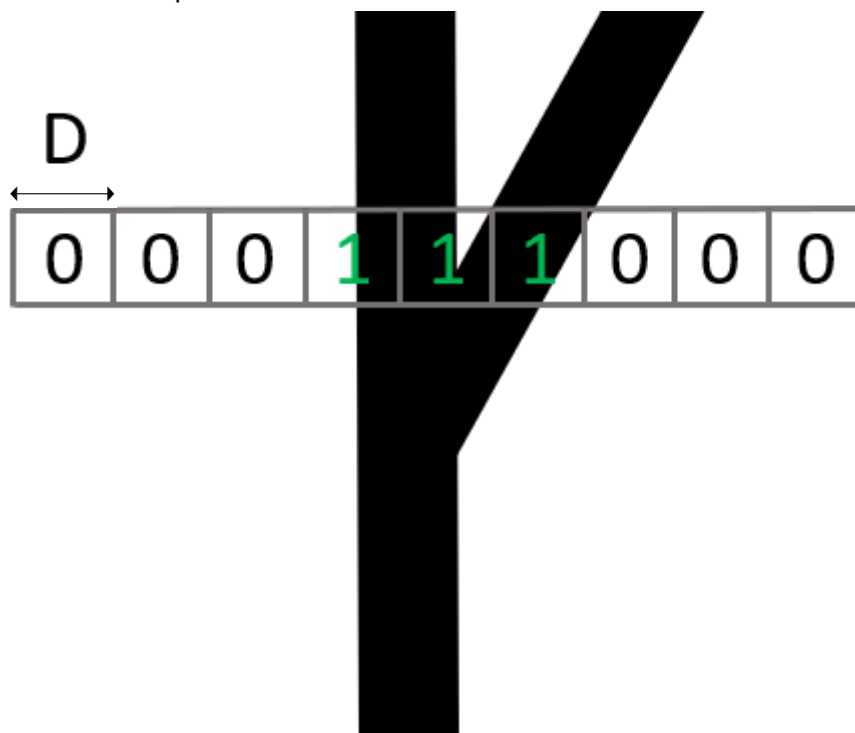
Hallův jev funguje na principu vychýlení elektronů za působení magnetického pole. Pokud vodičem prochází proud I a na vodič působíme magnetickým polem B , elektrony jsou nuceny k vychýlení na jednu stranu vodiče, tato nerovnováha vytvoří tzv. Hallovo napětí U . Hallovy sondy jsou schopné měřit velikost a polaritu Hallova napětí.

Magnetické pásky pro detekci trajektorie se prodávají v rolích a mají samolepící povrch. Magnetický pásek lze jednoduše přilepit na povrch, je to pasivní prvek, nezáleží na jeho znečištění a lze detekovat i skrz nekovové povrchy.



Obr. 1-7 Magnetický senzor MGS1600C [9]

Senzory pro magnetickou detekci trajektorie sdružují několik hallových senzorů pro přesnější detekci trajektorie. Existují specializované firmy jako je např. firma Roboteq [10], která poskytuje senzor MGS1600. (OBR. 1-7) Tento senzor má rozlišení $D = 1 \text{ mm}$, dokáže detekovat až 160 mm do šířky a mezi výškami od 10 do 60 mm od povrchu.



Obr. 1-8 Princip detekce trajektorie

Micro-senzory pro detekci jsou rozmístěné na jedné linii a tvoří jeden senzor pro detekci trajektorie. Podle jejich počtu jsme schopni zjistit rozlišení D daného senzoru. (OBR. 1-8) Čím větší je rozlišení daného senzoru, tím přesněji jsme schopni danou trajektorii detekovat. Samotný výstup ze senzoru, může být analogový, kdy pro každý micro-senzor senzor realizuje výstup v rozmezí (např. 0-5V), v závislosti na velikosti magnetického pole pod microsenzorem. Díky tomu dostáváme list hodnot v daném rozmezí, které musí řídicí systém digitalizovat. Pokud je výstup ze senzorů digitální (hodnoty 0|1), řídicí systém nemusí vstup ze senzoru digitalizovat.

1.7.2 Detekce pomocí indukce

V tomto případě se trajektorie zaznačí vodivým drátem, který se připevní na povrch a napojí na elektrický proud.

Pro detekci za pomoci indukce je nutné, aby byl drát zapojen na střídavý elektrický proud. Při střídavém zapojení proud mění směr, toto způsobuje i změnu magnetického pole okolo vodiče. Pokud do okolí této změny přiblížíme cívku, indukuje se do ní elektrický proud. Tento elektrický proud lze detekovat a v závislosti na jeho velikosti, jsme schopni určit zda se robot nachází nad trajektorií či ne.

Drát, kterým prochází proud, lze také detekovat pomocí Hallovy sondy, protože průchozí proud vyvolává magnetické pole okolo vodiče.

Výhody toho zapojení jsou v možnosti detekce přes nekovové plochy. Hlavní nedostatek této metody je v nutnosti napájení drátu.

1.7.3 Vizualní detekce

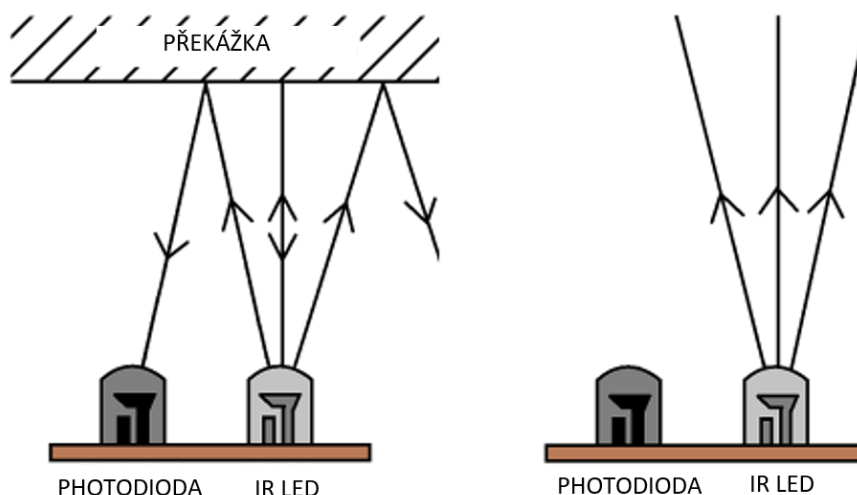
Vizualní detekce trajektorie se nejčastěji řeší pomocí photo-coupleru, nebo kamery. Při zaznačení trajektorie dbáme na kontrast podkladu a samotné trajektorie. Značení trajektorie provádíme 2 způsoby a to bílý podklad a černá trajektorie, nebo černý podklad a bílá trajektorie. Toto zaznačení je velmi praktické, protože lze provést barvou, nebo černou páskou, lze aplikovat na většinu povrchů a snadno upravovat. Hlavní problém, který při vizualní detekci může nastat, je nepřesný optický vstup do detekujícího zařízení. Nepřesný optický vstup může způsobit např. odřená barva na trajektorii, nebo změna světelných podmínek.

1.7.3.1 Detekce pomocí photo-coupleru

Photo-coupler je pár infra led emitting diode (diody, která emituje infra-červené záření), dále jen IR LED a fotodiody. Princip tohoto měřicího páru spočívá v emitování IR záření za pomoci IR LED a jeho následnou detekcí fotodiodou. Dle intenzity záření a způsobu zapojení jsme schopni změřit například reflexi povrchu, nebo detekci překážky.

Existují dva typy zapojení photo-coupleru přímé a nepřímé. Přímé zapojení je zapojení IR led a fotodiody naproti sobě. Toto zapojení se používá například pro detekci překážky mezi tímto párem. IR LED emituje záření, které fotodioda detekuje. Při vložení reflektivního či pohlcujícího materiálu je toto spojení mezi párem přerušeno. Díky tomu jsme schopni detekovat překážku mezi tímto párem.

Nepřímé zapojení používá podobný princip jako přímé, opět se jedná o signál za pomoci IR záření. Rozdíl oproti přímému je v postavení IR LED a fotodiody. (OBR. 1-9) Tyto prvky jsou postavené vedle sebe v těsné blízkosti. IR LED vysílá záření, pokud se před těmito dvěma prvky objeví překážka, která dokáže odrážet IR záření, záření je zpět odraženo a fotodioda ho zachytí. Pokud před photo-couplerem není žádná překážka, nebo pokud překážka pohlcuje záření, na fotodiodu se neodráží IR záření. Nepřímé zapojení umožňuje detekci změny síly záření v závislosti na povrchu překážky a zároveň v závislosti na vzdálenosti objektu.



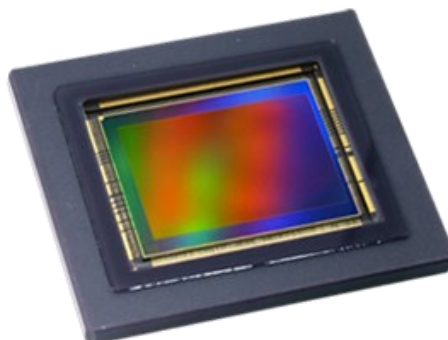
Obr. 1-9 Nepřímé zapojení photo-coupleru

Pro detekci trajektorie lze použít nepřímé měření. Černý povrch vysoce pohlcuje záření, tzn. že pokud bude měřící řetězec nad černou trajektorií IR záření nebude detekováno. Kontrastní povrch například bílý, bude odrážet IR záření.

Infračervené senzory jsou jedny z nejpoužívanějších komerčních způsobů detekce trajektorie. Přesnost těchto senzorů závisí na počtu micro-párů photo-couplerů. Podle počtu těchto micro-párů pro danou šířku senzoru jsme schopni určit rozlišení senzoru D. (OBR. 1-8) Rozdíl mezi analogovým a digitálním vstupem je stejný jako u magnetického senzoru trajektorie. Analogové hodnoty v tomto případě micro-párů se budou měnit dle intenzity IR.

1.7.3.2 Detekce pomocí kamery

Kamery fungují za pomoci obrazového snímače (image sensor). Tento senzor se skládá z velkého počtu fotodiod, které detekují světelné záření. Dle intenzity záření fotodioda produkuje proud, který senzor převede na jasovou informaci pro daný bod obrazu. Kombinací informací ze všech fotodiod vznikne obrazový záznam. Analogové kamery poskytují výstup obrazu bez digitalizace. Analogové kamery mívají menší poměr rozlišení a snímkovací frekvence než digitální kamery, ale jsou levnější.



Obr. 1-10 Obrazový snímač [11]

Výstup analogové kamery musí řídicí systém digitalizovat a následně vyhodnotit. Detekci trajektorie, lze provést například pomocí digitálního zpracování obrazu. Princip je úprava vstupního obrazu a následná detekce trajektorie pomocí algoritmů např. Houghova transformace.

Existuje také možnost detekce a řízení robota na trajektorii pomocí neuronových sítí. Pomocí vstupních (obraz) a výstupních parametrů (výstup na motory) je síť podle vstupních hodnot obrazu z kamery schopna správně rozhodnout a korigovat robota tak, aby sledoval trajektorii.

1.7.4 Umístění senzorů

Typické umístění magnetického, nebo IR senzoru je na spodní části podvozku robota. Senzor musí být umístěn v dostatečné vzdálenosti od snímacího místa. Vzdálenost od měřicího místa je daná výrobcem daného senzoru. Umístění kamery závisí na typu kamery a jejího rozlišení. Pro menší rozlišení se snažíme kameru umístit ve větší blízkosti ke sledované trajektorii. Dále je při umístění kamery důležité dbát na úhel pod, kterým kamera trajektorii snímá. Podle úhlu mezi osou, která prochází ohniskem kamery, a snímaným povrchem upravujeme záběr kamery. Pro další popis je při úhlu 0° osa vycházející z ohniska rovnoběžná s rovinou snímaného povrchu a zvyšováním hodnoty úhlu nakláníme kameru směrem k povrchu. Pokud chceme zvýšit dohled robota budeme volit menší úhel naklonění kamery, kdy minimální úhel závisí na výšce, ve které se kamera nachází. Nejmenší dohled kamery bude, pokud úhel bude 90° , tzn. kolmost vůči rovině snímaného povrchu.

1.7.5 Srovnání typu senzorů detekce trajektorie

Technika	Jednodu. práce s trajektorií	Pasivní	Vizuálně nezávislé	Detekce skrz překážky
Magnetické	Ano	Ano	Ano	Ano
Indukce	Ne	Ne	Ano	Ano
Vizuální	Ano	Ano	Ne	Ne

Tab. 1-7 Srovnání typu senzorů detekce trajektorie

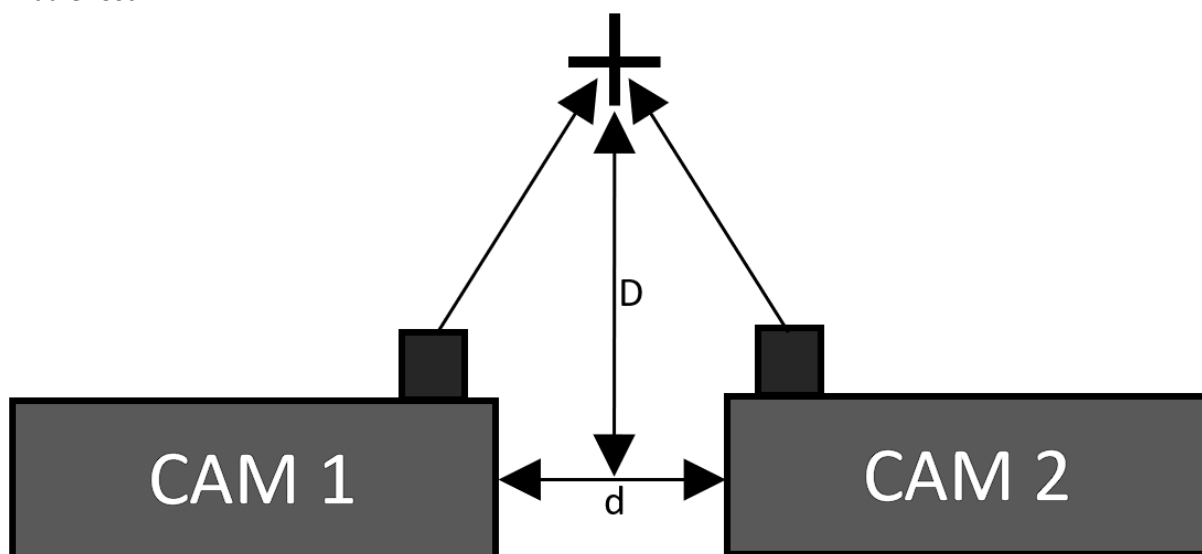
Hlavní problém indukce je v nutném napájení vodivého drátu a ve složitosti úpravě trajektorie, kdy je vodivý drát pevně zapuštěn do podlahy. Oproti tomu magnetické nebo vizuální zaznačení spočívá pouze v nalepení pásku na podlahu. Magnetická detekce a detekce za pomoci indukce nezávisí na znečištění trajektorie, nebo překážce mezi senzorem a samotnou zaznačenou trajektorií.

1.8 Senzory detekce překážek

Robot může při sledování trajektorie narazit na překážku. Pokud chceme, aby byl robot schopný s překážkou počítat je nutné ho vybavit senzorem pro její detekci a případné objetí. Při detekci překážky je nutné počítat s různým typem překážek, změnou světelných podmínek a jiných zásahů do detekce. Z toho důvodu se většinou používá kombinace různých senzorů tak, aby nedostatky jednoho typu kryly výhody jiného senzoru, tomuto postupu se říká „Fusion“. Senzory pro detekci překážek se dají rozdělit na pasivní a aktivní. Pasivní senzory pouze přijímají signál, detekují např. elektromagnetické záření (světlo). Aktivní senzory vysílají signál a následně signál přijímají. Dle rozdílu mezi vyslaným a přijatým signálem dokážou detekovat vzdálenost od překážky. Nejpoužívanější pasivní senzor je kamera. Nejpoužívanější aktivní senzory jsou LiDAR a sonar.

1.8.1 Kamera

Zjištění vzdálenosti od překážky se provádí pomocí triangulace. (1.7.3.2) Pokud požadujeme 3D detekci, je nutné použití více kamer pro vytvoření tzv. stereo systému. Základní stereo systém se skládá ze 2 kamer. (OBR. 1-11) Díky vzdálenosti d mezi kamerami a odlišností na obrazu můžeme spočítat vzdálenost D měřeného bodu. Pokud tyto systémy jsou správně zkalibrovány poskytují dobrý odhad vzdálenosti.



Obr. 1-11 Stereo kamerový systém

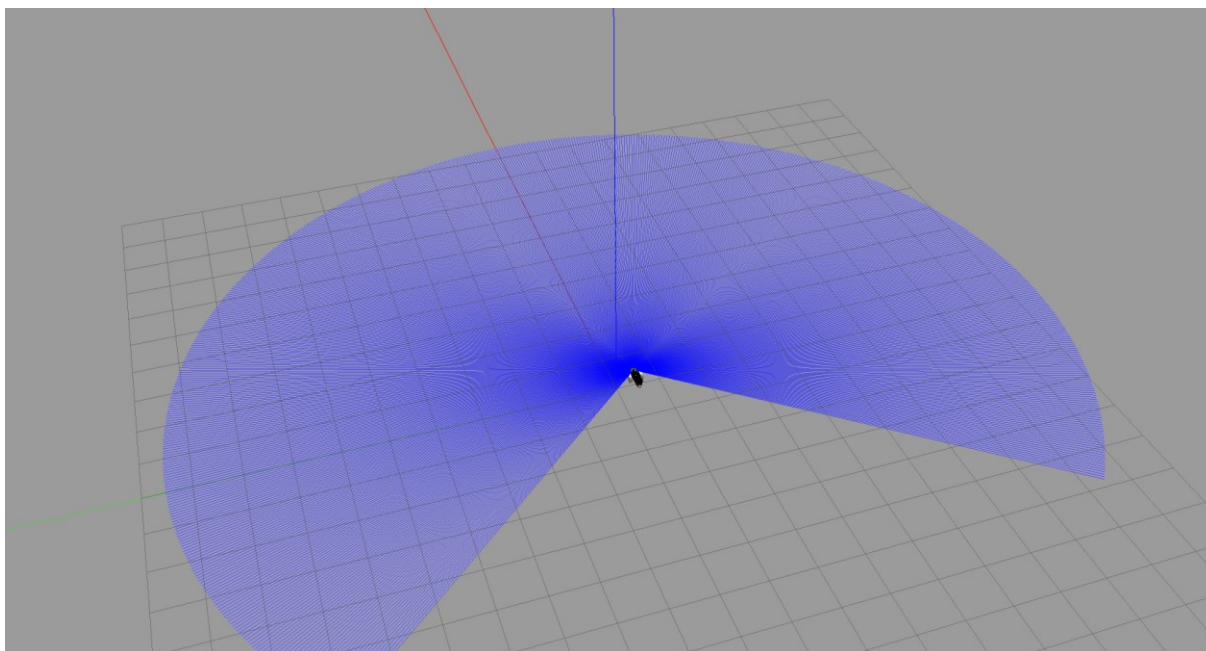
Podle prostředí, ve kterém bude detekce probíhat lze zvolit určitý typ kamery. Existují kamery pro noční vidění, nebo kamery které dokážou detekovat infračervené záření.

1.8.2 LiDAR (Light Detection and Ranging)

Lidar, je senzor, který vysílá krátké světelné impulzy za pomoci laseru a snímá čas za který se odražené světlo vrátí zpět k senzoru. Vzdálenost dané překážky se dá spočítat za pomoci vzorce.

$$vzdálenost = \frac{rychlost\ světla * čas}{2}$$

LiDAR má obvykle jeden laser, který se v průběhu měření otáčí. Díky tomu jsou tyto senzory schopny měřit v různém úhlovém rozsahu, který je daný výrobcem. Existují 2D, nebo 3D lidary. Lidary 2D dokážou snímat pouze v jedné rovině. (OBR. 1-12)



Obr. 1-12 LiDAR v simulačním softwaru Gazebo

Lidary nejsou přesné při dešti, sněhu a mlze a při jakékoliv optické bariéře. Lidary jsou oproti radarům výrazně levnější.

1.8.3 Sonar (Sound Navigation and Ranging)

Sonar vysílá zvukové pulsy za vysílače zvukových vln a snímá čas za který se zvuk odrazí zpět na přijímač. Primární použití těchto senzorů je pro detekci ve vodním médiu, ale v dnešní době se používají i pro detekci na vzduchu. Akustické frekvence těchto senzorů se pohybují mezi infrasonickými a ultrasonickými. Vzdálenost od překážky se dá spočítat za pomoci vzorce.

$$vzdálenost = \frac{rychlost\ zvuku * čas}{2}$$

Přesnost sonaru, závisí na prostředí, ve kterém se používá. Pokud sonar použijeme pro vodní detekci, bude mnohem přesnější než ve vzduchu. To je způsobeno rozdílem mezi hustotou vzduchu a vody. Ve vodě se zvuk šíří cca 4,3x rychleji než na vzduchu. Pokud sonar použijeme pro detekci překážek na vzduchu, sonar dokáže proniknout přes optické bariéry jako je mlha, či prach a zároveň dokáže

detekovat předměty, přes které prochází světlo. Zároveň je ale sonar nepřesný a závislý na teplotě vzduchu. Proto se sonary na vzduchu často používají jako kombinace s jiným typem senzoru.



Obr. 1-13 Ultrazvukový senzor HC-SR04

1.9 Pohybové ústrojí

Pohybové ústrojí musí umožňovat pohyb tak, aby umožnil sledování trajektorie. Průběh trajektorie se v čase mění, z toho vyplývá, že se robot musí změně přizpůsobit a natáčet se. Pro kontrolované zatáčení se nejčastěji používá kolový, nebo pásový podvozek.

1.9.1 Konstrukce podvozku

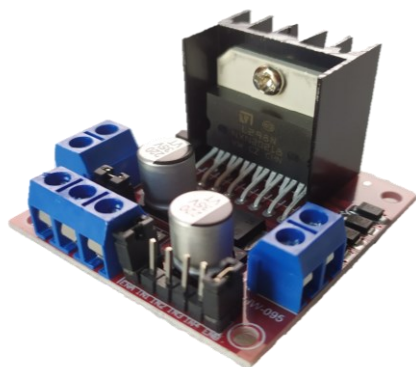
Nejčastější konstrukce pro soutěžní roboty jsou 2 kola s pohonem a 1 volné kolo, nebo caster wheel. Kuličkové kolečko zajišťuje stabilní podvozek při zatážení. Poháněné kola a většina hardwaru robota se nachází v zadní části robota, na přední části podvozku se nachází 1 volné kolo a senzory pro detekci trajektorie. Díky této konstrukci je robot schopný rychleji reagovat na změnu trajektorie.

Industriální roboti sledující trajektorii používají 4 kola s pohonem, nebo pásy. Tato konstrukce je výhodnější z důvodu lepšího rozložení zatížení, neboť většina těchto robotů slouží k převážení nákladu.

Důležitý parametr kol je jejich povrch. Pro robota, chceme docílit co nejlepší přilnavost vůči povrchu, aby nedocházelo k prokluzu kol. Z toho důvodu je lepší volit kola, které mají gumový povrch.

1.9.2 Pohon

Nejčastější pohonné jednotky pro roboty jsou elektrické motory. Elektrické motory se rozdělují na DC (stejnoseměrné) a AC (střídavé).



Obr. 1-14 H-bridge driver pro DC motor

Parametr	Značka	Jednotka
Výkon	P	[W]
Příkon	P_r	[W]
Účinnost	η	[1]
Otáčky	n	[ot·min ⁻¹]
Krouticí moment	M _k	[N·m]
Napětí	U	[V]
Proud	I	[A]

Tab. 1-8 Hlavní parametry motorů

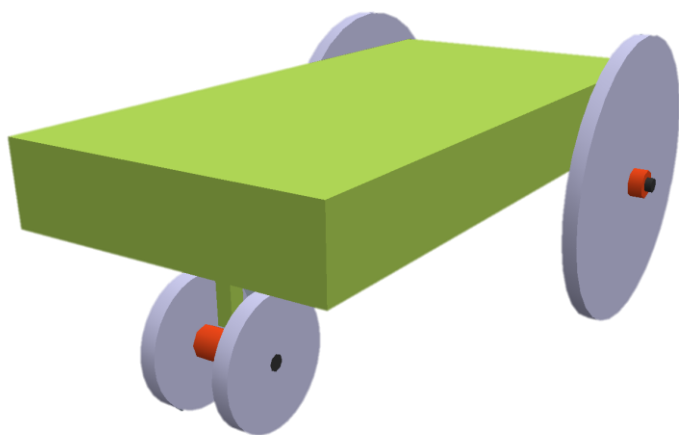
V závislosti na použití daného robota volíme typ motoru. Pro industriálního robota, který bude převážet těžké náklady volíme motor s větším kroutícím momentem, popř. musíme použít převodovku. Pro soutěžní roboty, které jsou lehké a potřebují dosáhnout co největší rychlosti volíme motory s vysokými otáčkami.

Elektrické motory se nejčastěji ovládají vstupním napětím, nebo proudem. Pro tyto ovládání existují drivery např. H-Bridge pro DC motor, které jsou určené pro určitý typ motoru. (OBR. 1-14) Tyto drivery mají napájecí vstupy pro motor a kontrolní vstupy, pomocí kterých ovládáme rychlost a směr otáčení motoru.

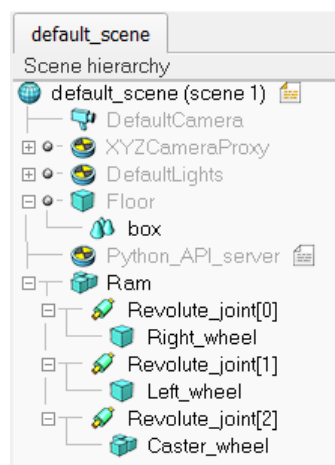
1.10 Simulace v CoppeliaSim

Model robota v CoppeliaSim záleží na vlastních komponentech a jejich vzájemných interakcích. Podle typu komponentu volíme typ objektu scény (TAB. 1-3).

Hierarchie mobilního robota souvisí s vazbami mezi komponenty. Mobilní 2 kolový robot (OBR. 1-15) je reprezentován hierarchií (OBR. 1-16). Primární shape „Ram“ je v pozici XYZ a v rotaci okolo těchto os vůči nulové pozici světa. Vůči tomuto objektu je ve vazbě kloub, který je ve vazbě s kolem „Left_wheel“. Stejný postup je u kloubu a vazbě druhého kola. Dále je „Ram“ ve vazbě s kloubem a kloub je ve vazbě s „Caster_wheel“.



Obr. 1-15 Jednoduchý model v CoppeliaSim



Obr. 1-16 Hierarchie mobilního robota

1.10.1 Shape

Shape simuluje 3d model komponentu. Shape může být nastaven jako kinematický=statický, nebo dynamický. Statický objekt nezahrnuje dynamické vlastnosti objektu. V závislosti na složitosti daného komponentu je doporučeno použít při simulaci zjednodušený tvar např. kvádr, krychle, válec nebo

koule a na tento jednodušší tvar zavazbit reálný model. V tomto případě simulace nebude požadovat pro výpočetní procesy tak velký výkon.

1.10.2 Joint

Joint symbolizuje pohybové spojení mezi 2 objekty. CoppeliaSim obsahuje rotační, prizmatický a sférický kloub. V závislosti na typu spojení volíme kloub a následně provádíme jeho nastavení. Kloubu lze nastavit úhlové/vzdálenostní omezení, maximální sílu/moment, způsob jeho ovládání a mód. Kloub lze nastavit do 3 módů kinematic, dynamic, dependent. Kinematic mód se používá pro přímou kontrolu kloubu, bez vlivu dynamiky. Tzn. že pokud systém od kloubu požaduje jiný úhel, v simulaci se provede okamžitá změna pozice. Dynamic mód zahrnuje dynamiku součástí mezi kterými je kloub vsazen. Při dependent módu závisí pozice kloubu na pozici jiného dle zadané rovnice, tento mód nezahrnuje dynamické účinky.

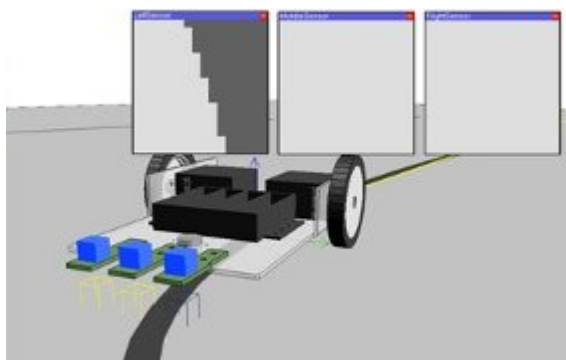
1.10.3 Způsob detekce trajektorie v simulačním prostředí

CoppeliaSim obsahuje tzv. „Vision“ senzory. (OBR. 1-17) Tyto senzory dokážou simulovat kamerový senzor či photo-coupler. Existují dva typy těchto senzorů, a to senzory s ortogonálním pohledem, nebo pohledem v perspektivě. U vision senzorů jsme schopni nastavit jejich rozlišení, či prostor který snímají. V závislosti na senzoru, který má simulace reprezentovat volíme jeho nastavení.

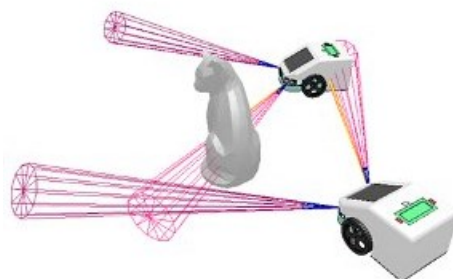
1.10.4 Způsoby detekce překážek v simulačním prostředí

Simulační software CoppeliaSim obsahuje tzv. „Proximity“ senzory. (OBR. 1-18) Tyto senzory dokážou detekovat vzdálenost od měřených objektů, pokud daný objekt tuto detekci umožňuje. Proximity senzor se definuje dle senzoru, který se v simulaci nahrazuje. Dle typu senzoru se volí detekční objem a nastavuje se způsob detekce objektu.

Proximity sensory můžou simulovat prakticky jakýkoliv typ senzoru vzdálenosti jako jsou infračervené, ultrazvukové či radarové senzory.



Obr. 1-17 Vision senzory [12]



Obr. 1-18 Proximity senzory [13]

1.11 Algoritmus pro sledování trajektorie

Algoritmy pro sledování trajektorie fungují na principu detekce vzdálenostní odchylky středu robota vůči pozici detekované trajektorie. V závislosti na typu senzoru lze vypočítat pozici trajektorie a následně rozdílem vůči pozici robota, které chceme dosáhnout tuto pozici odečteme. Tímto získáme odchylku, kterou regulujeme.

Pokud budeme předpokládat, že se pozice trajektorie pohybuje v určitém intervalu, který je symbolizovaný váhovou pozicí senzoru, lze v závislosti na naměřených hodnotách jednotlivých měřících párů celého senzoru vypočítat pozici trajektorie. (OBR. 1-8)

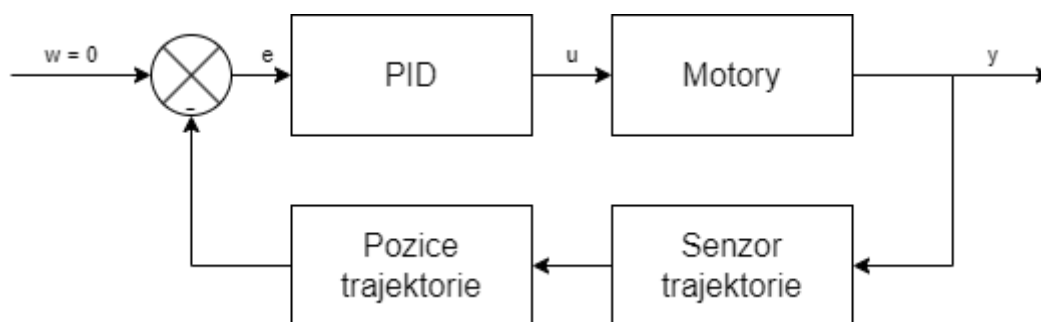
Výpočet pozice lze realizovat pomocí váženého průměru. [14]

$$e = \frac{\sum_{i=0}^n x_i \cdot y_i}{\sum_{i=0}^n y_i}$$

x_i = váha pozice senzoru

y_i = naměřená hodnota senzoru

Následně se jedná o regulaci poziční odchylky, kdy požadovaná hodnota je pozice měřícího páru, který se nachází uprostřed senzoru. Toto symbolizuje pozici robota, které chceme dosáhnout. Tato regulace se dá symbolizovat regulačním schématem. (OBR. 1-19)



Obr. 1-19 Regulační obvod pro sledování trajektorie

Jako regulátor, lze použít např. regulátor typu PID, který lze realizovat ve scriptu simulačního softwaru.

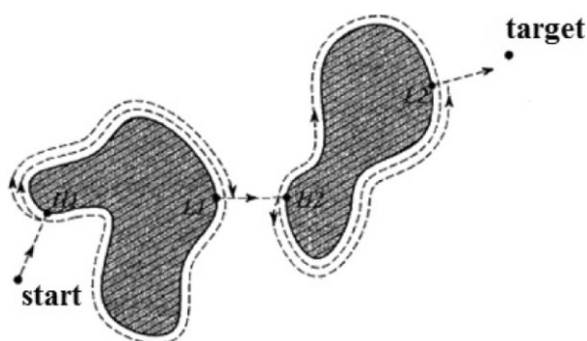
Dle výstupu regulátoru se realizuje akční zásah do pohybového ústrojí robota.

1.12 Algoritmy pro vyhýbání se překážkám

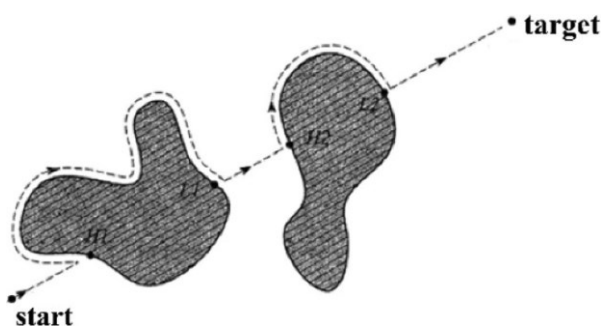
Algoritmy pro vyhýbání překážkám řeší problém při kterém robot na trase detekuje překážku. Nejjednodušší algoritmus porovnává vzdálenost mezi překážkou a robotem. Při naměření vzdálenosti menší než minimální nastavená, robot zastaví. Tento algoritmus, ale nezajišťuje doražení do cíle. Jedny z neznámějších algoritmů při kterých robot je schopný dosáhnout cílové pozice jsou algoritmy Bug či Bubble.

1.12.1 Bug algoritmy

Tyto algoritmy jsou jedny z nejjednodušších způsobů vyřešení překážky na trase robota. Podmínka pro tyto algoritmy je znalost startovní, aktuální a cílové pozice robota. Dále musí být robot schopný objíždět překážku. Existuje více typů bug algoritmů. Neznámější jsou typy Bug1 a Bug2. [15]



Obr. 1-20 Bug1 algoritmus [16]



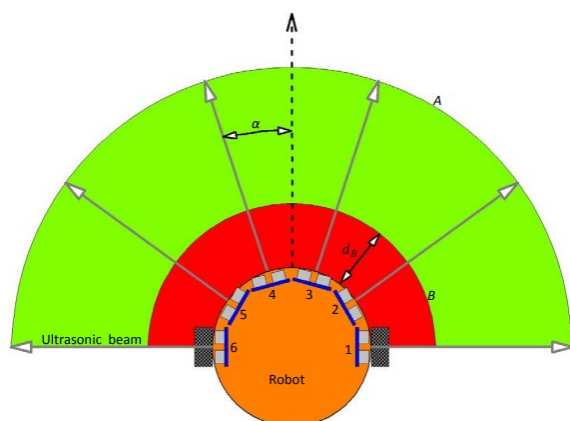
Obr. 1-21 Bug2 algoritmus [17]

Algoritmus Bug1 při detekování překážky, robot objede celou překážku a při objíždění detekuje nejkratší vzdálenost mezi danou pozicí a cílem trasy. (OBR. 1-20) Po objetí celé překážky se vrátí na pozici kde se vyskytovala nejmenší vzdálenost k cíli a vyrazí směrem k cíli.

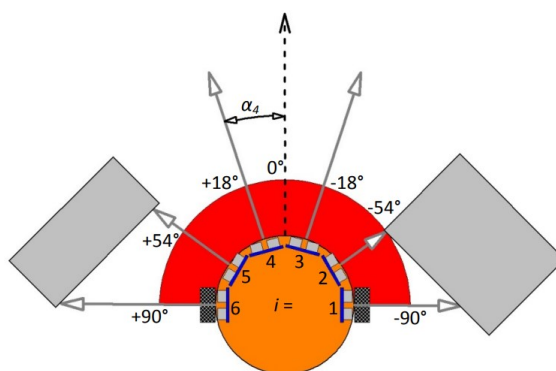
Algoritmus typ Bug2 nehledá minimální vzdálenost, ale detekuje úhel vůči finální pozici. Na startu algoritmu robot vypočítá směrový úhel vůči cíli. (OBR. 1-21) Při objíždění překážky porovnává aktuální úhel se startovním. Jakmile se tyto úhly rovnají vyrazí k cíli.

1.12.2 Bubble algoritmus

Bubble algoritmus pracuje s tzv. „bubble“, prostorem kolem robota. [18] Robot obsahuje větší počet senzorů, které jsou rozmístěné tak, aby dokázaly snímat vzdálenost vůči překážkám kolem robota. (OBR. 1-22) (OBR. 1-23) Pokud je detekovaná vzdálenost od překážky menší než nastavená, (d_B) provede se algoritmus pro vyhýbání překážkám. Sensory detekují vzdálenosti od překážek a robot se natáčí a pohybuje směrem kde detekuje nejmenší hustotu překážek. Toto provádí dokud není cíl trasy plně viditelný, nebo dokud nepřestane detekovat překážky.



Obr. 1-22 Bubble algorithmus 1 [18]



Obr. 1-23 Bubble algorithmus 2 [18]

2 Požadavkový list

Požadavkový list vychází z pravidel soutěží line followerů. [19] Robot musí, dle definovaných cílů práce, být schopen vyhýbat se překážkám. Z toho důvodu bylo nutné tuto funkci přidat do listu požadavků. Požadavkový list je rozdělen na 2 hlavní části, a to na hodnotové požadavky (TAB. 2-1) pro simulaci a robota a dále na softwarové funkce (TAB. 2-2), které spadají pod jednotlivé systémy.

		Parametr	Hodnota
Simulace		Simulační prostředí	CoppeliaSim
		Programovací jazyk	Python, Lua
	Trajektorie	Dynamická generace trajektorie	Ano
		Barva	Černá
		Rozměr	Šířka 2cm
		Tvar	Spojité křivky
		Min. zaoblení	Rádus 8cm
		Min. ostrý úhel	45 stupňů
		Překrytí	Ano
	Překážky	Dynamická generace překážek	Ano
		Rozměr	min 10x10x10cm
		Hmotnost	min 100g
	Podklad	Barva	Bílá
Robot		Max. rozměr	25cm délka, 25cm šířka, 20cm výška
		Max. hmotnost	0,5kg
		Počet kol	2 + caster wheel
		Mikrokontroler	Arduino
	Senzory	Senzor trajektorie	Analogový infračervený
		Senzor překážek	Analogový infračervený
		Počet senzoru překážek	3
	Motory	Počet motorů	2
		Typ motorů	DC
		Motor driver	DC pro 2 motory

Tab. 2-1 Hodnotové požadavky

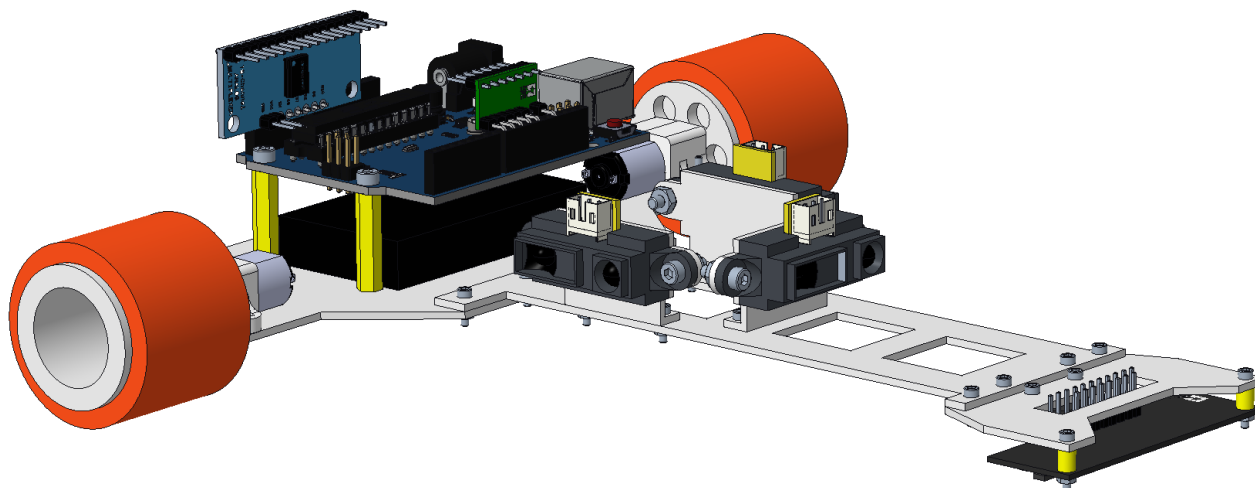
			Funkce
Software	Simulace		Algoritmus generace trajektorie
			Algoritmus generace překážek
			Automatický restart
	Robot	Detekční systém	Čtení dat ze senzoru trajektorie
			Čtení dat ze senzoru překážek
			Komunikace s kontrolním systémem
		Řídicí systém	Rozhodovací algoritmus
			Algoritmus sledování trajektorie
			Algoritmus řešení překážek
			Komunikace se pohybovým ústrojím
		Pohybový systém	Řízení motorů
			Výstup pozice z enkodérů

Tab. 2-2 Softwarové funkce

3 Praktická část

Simulace vychází z podrobného modelu robota a reálných dostupných komponent. Robot musí jít jednoduše sestavit a obsahovat prostor pro vedení kabelů, tak aby nezasahovali do FOV senzorů. Simulační model robota musí být schopen sledovat trajektorii. Při detekci překážky na trajektorii tuto překážku objet a dostat se zpět na sledovanou trať. Samotná simulace má obsahovat dynamický generátor trajektorie a překážek. Tento generátor jedná v závislosti na pozici robota v simulaci.

3.1 Model robota



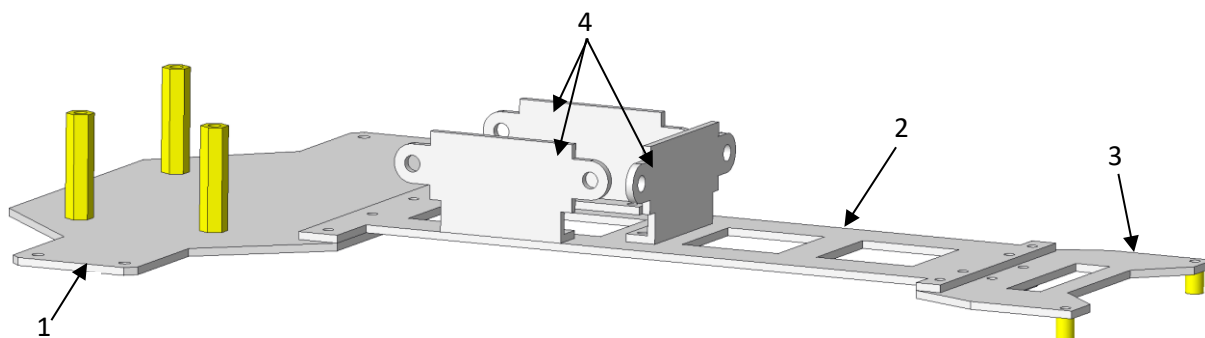
Obr. 3-1 Model robota

Při tvorbě modelu robota, jsem vycházel z požadavkového listu a z konstrukcí soutěžních robotů. Konstrukce vychází z maximálních rozměrů, které jsou definované v požadavkovém listu a dále ze spojení jednotlivých definovaných komponent. Model byl vytvořený v systému Creo Parametric.

3.1.1 Nosná konstrukce

Nosná konstrukce musí umožnit montáž kol, elektroniky a vedení napájecích i signálních rozvodů, bez zásahu do FOV jednotlivých senzorů. Nosná konstrukce se skládá ze 3 desek a ze tří L konzol. (OBR. 3-2) Tyto prvky lze vyrobit pomocí 3d tiskárny z materiálu PETG či PLA.

První deska (1) umožňuje montáž elektromotorů a mikrokontroleru Arduino, které je vyvýšené pomocí distančních sloupků. Díky tomuto vyvýšení lze mezi desku a Arduino vložit baterii.



Obr. 3-2 Nosná konstrukce

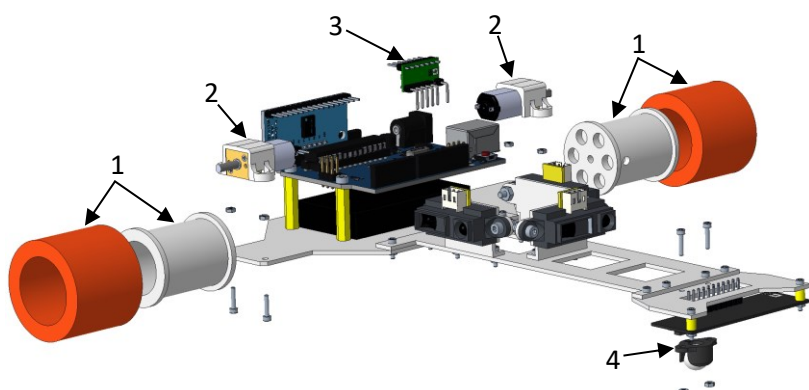
deska 1-č.1, deska 2-č.2, deska 3-č.3, L-konzoly-č.-4

Druhá deska (2) spojuje zadní a přední desku, pomocí spojovacích šroubů a matic. Vybrané profily v této desce slouží k vedení rozvodů a zároveň snižují celkovou hmotnost konstrukce. Dále jsou na této desce L konzoly (4) na kterých jsou následně přimontovány jednotlivé senzory vzdálenosti.

Deska, která se nachází v přední části robota (3) umožňuje montáž caster wheel a zároveň pomocí distančních prvků montáž senzoru trajektorie.

3.1.2 Lokomoční subsystém

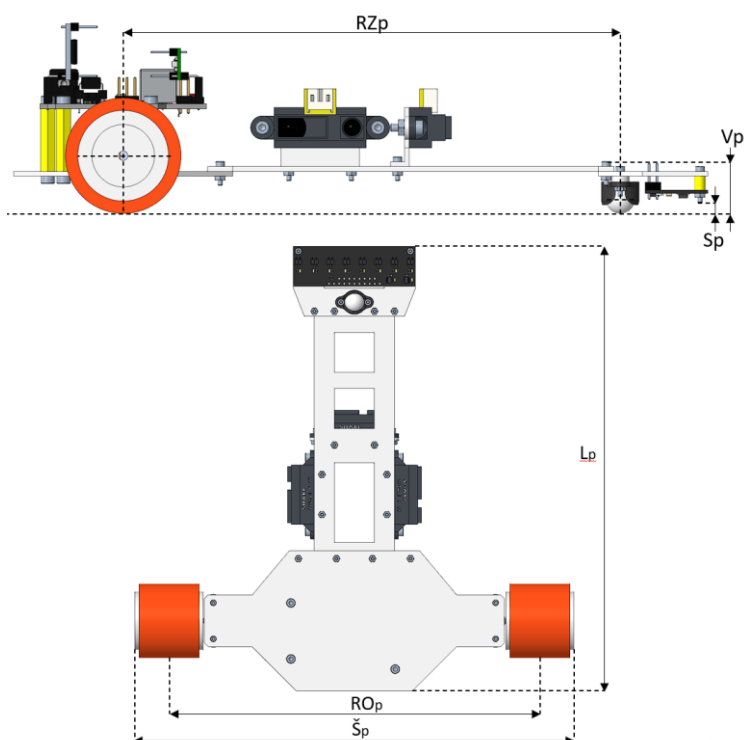
Lokomoční subsystém se skládá ze 2 kol, 1 caster wheel, DC motorů a driveru pro motory. Jednotlivé komponenty lze vidět na (OBR. 3-3), nejdůležitější parametry podvozku jsou uvedeny v tabulce (TAB. 3-2).



Komponent	Poz.
Kola JSumo JS2622	1
Micro Metal Gearmotor	2
DRV8835 Motor driver	3
Caster wheel 3/8" Plastic	4

Obr. 3-3 Lokomoční subsystém

Tab. 3-1 Lokom. Subsystém komponenty



Parametry	Zkratka	[mm]
Výška podvozku	Vp	16,6
Šířka podvozku	Šp	219,3
Délka podvozku	Lp	222
Světlost podvozku	Sp	3,5
Rozpor podvozku	RZp	159
Rozchod podvozku	ROp	185,3

Obr. 3-4 Lokomoční subsystém pohledy

Tab. 3-2 Lokom. Subsystém parametry

3.1.2.1 Motory

Motory volím Micro Metal Gearmotor HPCB 6 V od výrobce Pololu. (OBR. 3-5) Jedná se o výkonné stejnosměrné motory s vestavěnou převodovkou. (TAB. 3-3) Výhoda těchto motorů je ve velkých možnostech výběru dle momentu a otáček při zachování původních rozměrů motoru. Z toho důvodu je jednoduché vyměnit případně motor za jiný typ. K těmto motorům lze zároveň koupit

přípevňovací objímku. Existuje verze tohoto motoru s dvojitou hřídelí, která slouží pro vstup do enkodéru.



Obr. 3-5 Motor [20]

Micrometal Gearmotor HPCB 6V		
Parametr	Hodnota	Jednotka
Převodový poměr	297.92:1	[1]
No-load rychlost	110	rpm
Stall proud	1,5	A
Stall moment	3,4	kg*cm
Maximální účinnost	0,33	[1]
Hmotnost	9,5	g

Tab. 3-3 Parametry motoru [20]

3.1.2.2 Driver pro motory

Driver vybírám DRV8835 Dual Motor Driver Carrier (OBR. 3-6) ze stránek Pololu. Jedná se o driver pro kontrolu 2 stejnosměrných motorů. (TAB. 3-4) K tomuto driveru lze jednoduše připojit standardní piny, tak aby kontrolní piny zasedly přímo do desky Arduino. Ovládání motorů pomocí tohoto driveru a Arduino lze zajistit pomocí oficiální knihovny k Arduino IDE od výrobce Pololu.



Obr. 3-6 Driver DRV8835 [21]

DRV8835 Dual motor driver		
Parametr	Hodnota	Jednotka
Počet vstupních pinů	4	[1]
Počet výstupních pinů	4	[1]
Napájecí napětí	0-11	V
Výstupní proud (na 1 motor)	1,2	A
Kontrolní napětí	2-7	V
Hmotnost	0,5	g

Tab. 3-4 Parametry DRV8835 [21]

3.1.2.3 Kola

Kola pro robota jsou od výrobce JSumo. (OBR. 3-7) Jedná se o kola JS2622 s hliníkovou vnitřní konstrukcí a silikonovou gumou. Tyto kola lze jednoduše namontovat na motory s průměrem hřídele 3 mm pomocí šroubu dodávaným přímo ke kolům. Kola váží pouze 13 g, umožňují jednoduchou montáž a jsou určené pro sumo roboty, nebo line follower roboty.

Caster wheel je vybráno od výrobce Pololu. (OBR. 3-8) Má lehkou konstrukci (0.8 g) a plastovou kuličku.



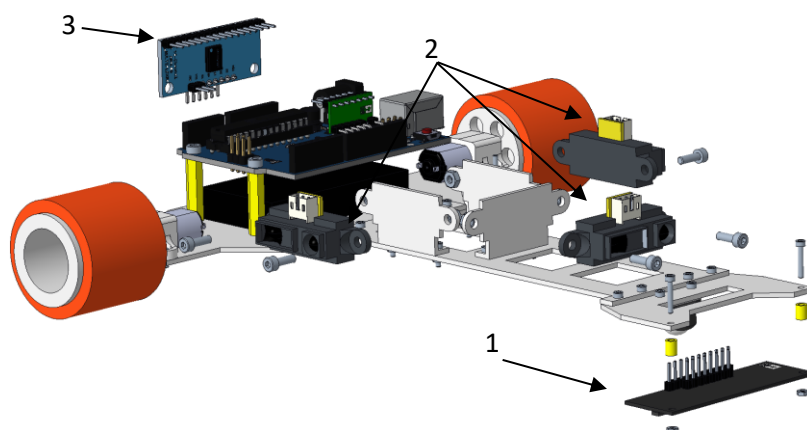
Obr. 3-7 Kola 1S2622 [22]



Obr. 3-8 Pololu Caster wheel [23]

3.1.3 Senzorický subsystém

Senzorický subsystém se skládá ze 3 senzorů vzdálenosti, 1 senzoru pro detekci trajektorie a multiplexeru. Jednotlivé komponenty lze vidět na Obr. 3-9.



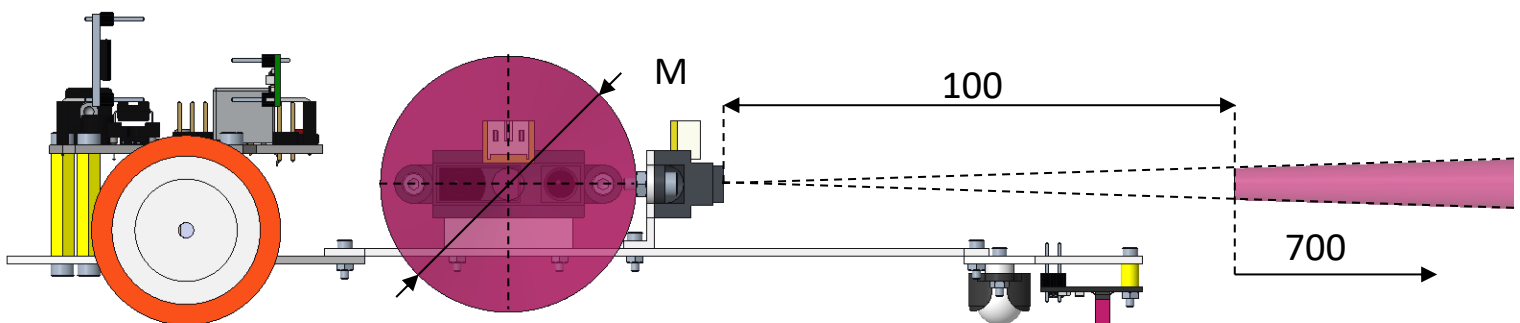
Obr. 3-9 Senzorický subsystém

Komponent	Poz.
QTR-MD-08A	1
Sharp GP2Y0A21YK0F	2
Multiplexer CD74HC4067	3

Tab. 3-5 Senzor. subsystém komponenty

3.1.3.1 Senzor vzdálenosti

Zvolený senzor vzdálenosti je GP2Y0A21YK0F od výrobce Pololu. Jedná se o infračervený analogový senzor pro detekci vzdálenosti od objektu mezi 10-80 cm. Na maximální měřicí vzdálenosti 80 cm má senzor 5 cm(M) rozsah pohledu. [24] Toto umožňuje jeho montáž v nižší poloze, než by byl například ultrazvukový senzor HC-SR04. (OBR. 3-10) Analogový výstup tohoto senzoru se pohybuje mezi 2,5-0,4 V v závislosti na detekované vzdálenosti od reflektivního objektu. Senzor obsahuje 2 vstupní piny pro napájení a 1 analogový výstupní pin. Robot obsahuje 3 tyto senzory natočené vzájemně o 90°.



Obr. 3-10 FOV Senzoru vzdálenosti



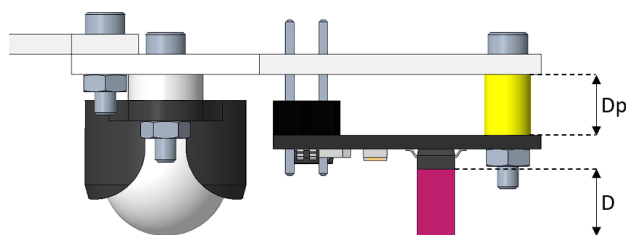
Obr. 3-11 GP2Y0A21YK0F [25]

GP2Y0A21YK0F		
Parametr	Hodnota	Jednotka
Počet výstupních pinů	1	[1]
Typ výstupu	Analog	[1]
Analogový signál	0,5-3	V
Range	10-80	cm
Napájecí napětí	4,5-5,5	V
Napájecí proud	30	mA
Hmotnost	3.5	g

Tab. 3-6 Parametry GP2Y0A21YK0F [25]

3.1.3.2 Senzor trajektorie

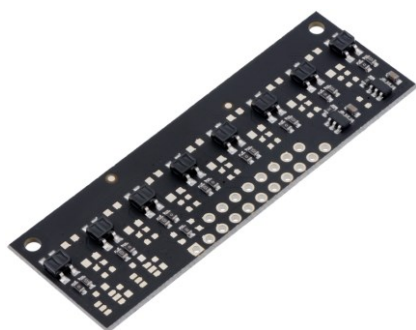
Zvolený senzor pro detekci trajektorie je infračervený senzor QTR-MD-08A. (OBR. 3-13) Tento senzor je z nové generace senzorů od výrobce Pololu. (TAB. 3-8) Jedná se o analogový senzor s 8 měřícími fototranzistory s možností nastavení napětí na IR diodách. Pro umístění v optimální výšce 5 mm od snímaného povrchu, bylo nutné distancovat tento senzor od nosné konstrukce pomocí distančních podložek. (OBR. 3-12)



Parametry	Zkratka	[mm]
Doporučená vzdálenost ST	D	5
Distanční podložka	Dp	4,5
Rozsah SV	-	100-800

Obr. 3-12 FOV senzoru trajektorie

Tab. 3-7 Parametry senzoru trajektorie



Obr. 3-13 QTR-MD-08A [26]

QTR-MD-08A		
Parametr	Hodnota	Jednotka
Počet výstupních pinů	8	[1]
Typ výstupu	Analog	[1]
Optim. vzdálenost od povrchu	5	mm
Napájecí napětí	2,9-5,5	V
Napájecí proud	125	mA
Vzdálenost mezi couplery	8	mm
Hmotnost	3,1	g

Tab. 3-8 Parametry QTR-MD-08A [26]

3.1.3.3 Multiplexer

Z důvodu nedostatku analogových pinů na desce Arduino je nutné použít multiplexer. (OBR. 3-14) Používaný komponent pro řešení tohoto problému je Analogový multiplexer pro 16 kanálů CD74HC4067. (TAB. 3-9) Tento komponent umožňuje zvýšení počtu analogových pinů pomocí připojení na 1 Analogový vstup na Arduino a 4 digitálních pinů. Díky tomuto multiplexeru jsme schopni zajistit připojení 16 analogových vstupů. Pro tento senzor existuje knihovna pro Arduino IDE.



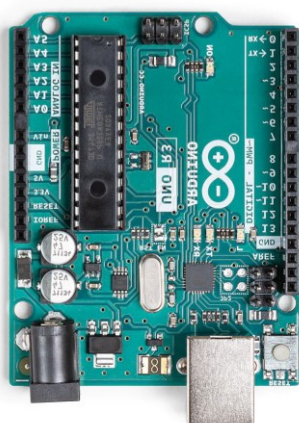
Obr. 3-14 MUX CD74HC4067 [27]

Multiplexer CD74HC4067		
Parametr	Hodnota	Jednotka
Počet kontrolních pinů	4	[1]
Počet vstupních pinů	12	[1]
Napájecí napětí	1,2-6	V
Hmotnost	1	g

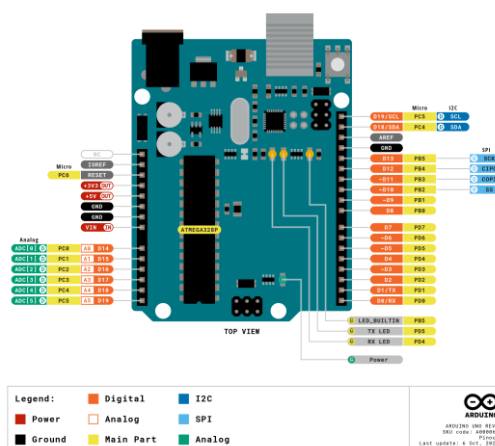
Tab. 3-9 Parametry MUX CD74HC4067 [27]

3.1.4 Mikrokontroler

Vybraný mikrokontroler je Arduino UNO. (OBR. 3-15) Jedná se o jeden z nejznámějších programovatelných mikrokontrolerů. Obsahuje 14 digitálních output/input pinů a 6 analogových vstupů. (OBR. 3-16) Vzhledem k potřebě minimálně 11 analogových vstupů ze 3 senzorů vzdálenosti a 1 senzoru trajektorie je nutné použít multiplexer. (3.1.3.3) Programování tohoto mikrokontroleru se realizuje pomocí Arduino IDE, který umožňuje přidání knihoven od oficiálních výrobců vybraných komponent. Výrobce doporučuje optimální napětí pro napájení v rozsahu 7-12 V což vzhledem k vybrané baterii vyhovuje. Deska obsahuje interní regulátor napětí. Teoretický možný výstupní proud z 5 V pinu z desky je 800 mA. Toto je v rozsahu napájení všech senzorů robota zároveň.



Obr. 3-15 Arduino UNO [28]



Obr. 3-16 Arduino UNO-pins [28]

3.1.5 Baterie

Baterie pro napájení robota je G3 Ray Li-Pol určená pro aplikaci v modelech letadel a vrtulníků. (OBR. 3-17) Při teoretickém maximálnímu proudu robota 3.2 A je tato baterie schopna zajistit provoz kolem 10 minut. (TAB. 3-10) Jedná se pouze o přibližnou dobu provozu, kterou nejvíce ovlivňuje maximální spotřeba motorů.



Obr. 3-17 Baterie G3 [29]

Baterie G3 Ray Li-Pol		
Parametr	Hodnota	Jednotka
Napětí	7,4	V
Nabíjecí proud	450-900	mA
Vybíjecí proud	13,5	A
Kapacita	450	mAh
Hmotnost	33	g

Tab. 3-10 Parametry Baterie G3 [29]

3.1.6 Spojovací komponenty

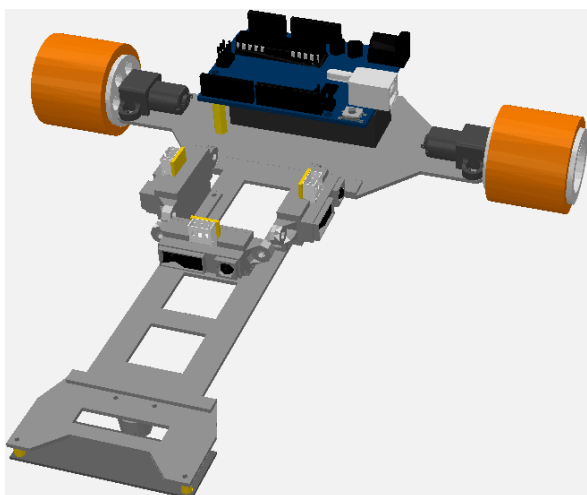
Spojení jednotlivých komponent je zajištěno pomocí dodaných šroubů k jednotlivým komponentům. Celková konstrukce je následně spojena pomocí šroubů s válcovou hlavou s vnitřním šestihranem normy DIN 912 a šestihranných matic normy DIN 934.

3.2 Simulace

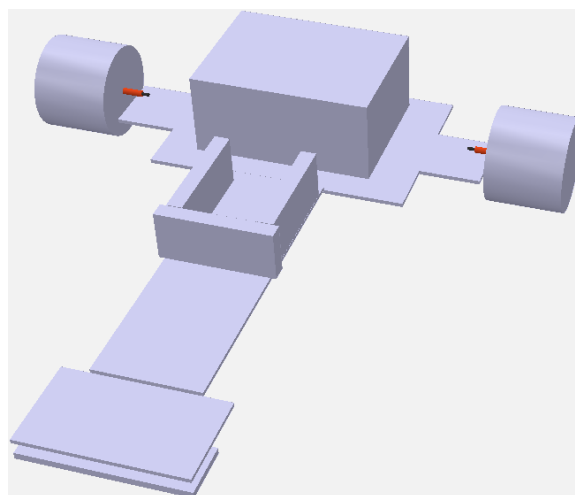
Simulace byla vytvořena v simulačním systému CoppeliaSim a model převeden ze systému PTC Creo Parametric.

3.2.1 Převedení do simulace

Jako první došlo k vytvoření simulačního modelu robota z exportovaných souborů ze systému Creo v podobě stl. Z důvodu complexity modelu a doporučení dle *Coppelia-help-files* [30] je nutno zjednodušit celkový model pomocí předdefinovaných funkcí např. Decimate, tak aby model obsahoval kolem 20000 trojúhelníků. Pokud by model obsahoval násobně větší počet, simulace by nebyla plynulá. Z tohoto důvodu je model v simulaci vidět zjednodušený, ale zanechává si dynamické parametry zjištěné z Creo Parametric. (OBR. 3-18) Nejdůležitější parametry jsou hmotnost a momenty setrvačnosti. Momenty setrvačnosti musí být definované ke stejnému souřadnicovému systému v jakém je daný objekt definovaný v simulaci. (TAB. 3-11) Pro zjištěných daných hodnot lze v systému Creo definovat daný souřadnicový systém.

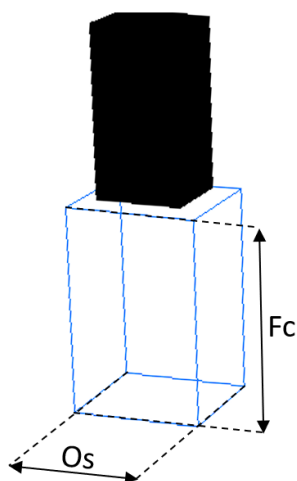


Obr. 3-18 Visuální model



Obr. 3-19 Dynamický model

Jednotlivé stl modely jsou připojené k jejich zjednodušeným verzím, díky kterým opět dochází ke zlepšení výkonu simulace. (OBR. 3-19)



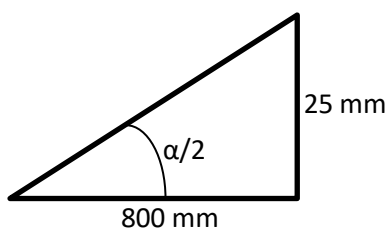
Obr. 3-20 Vision senzor

Parametry		Hodnota
Hmotnost	7,4	161,94 g
Momenty setrvačnosti	Ixx	207 kg·mm ²
	Iyy	897,2 kg·mm ²
	Izz	1070,9 kg·mm ²

Tab. 3-11 Parametry simulačního modelu

Jednotlivé senzory jsou nastavené dle známých parametrů daných výrobcem. Senzor pro detekci trajektorie QTR-MD-08A je v simulaci realizovaný pomocí osmi visual senzorů nastavených na detekci 1 pixelu. (OBR. 3-20) Nejdůležitější parametry jsou *Far Clipping plane* (Fc) nastavený na 5 mm a *Orth. size* (Os) nastaven na 3 mm.

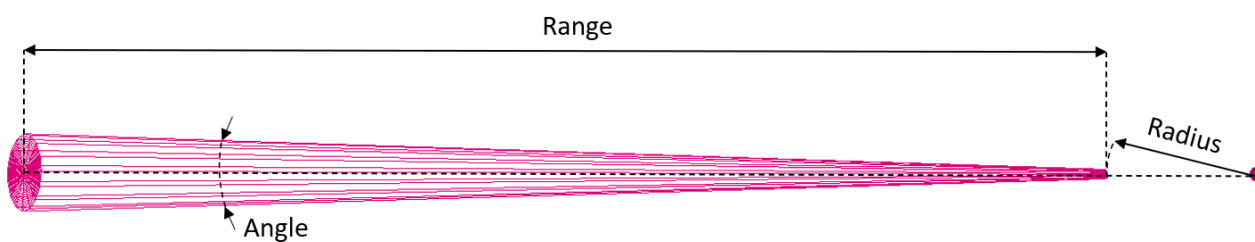
Senzory vzdálenosti GP2Y0A21YK0F jsou realizované pomocí proximity senzorů. Nejdůležitější parametry jsou range, úhel kužele a radius. (OBR. 3-23) Tyto parametry symbolizují výšku kužele, jeho rozpětí a oblast ve které senzor není schopen detekce. Výška kužele je dána výrobcem. Úhel kužele lze vypočítat (OBR. 3-21, OBR. 3-22) za pomoci parametrů známých z webu Solarbotics. [24]



Obr. 3-21 FOV senzoru GP2Y0A21YK0F

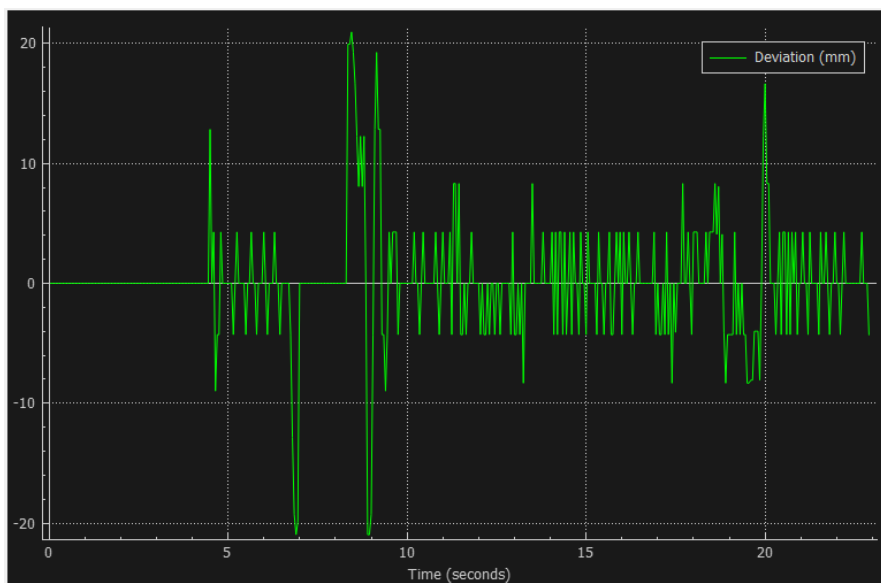
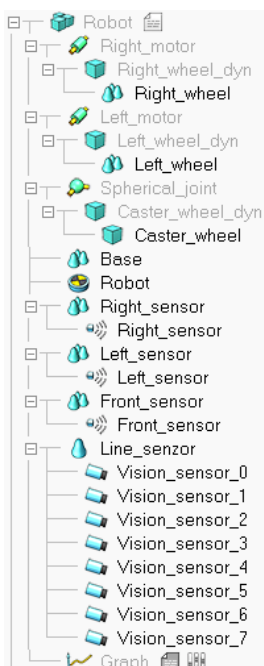
$$\alpha = 2 \cdot \tan^{-1} \left(\frac{25}{800} \right) = 3,58^\circ$$

Obr. 3-22 výpočet úhlu



Obr. 3-23 FOV proximity senzoru

Celková hierarchie robota lze vidět na obrázku. (OBR. 3-24) Pro vykreslení odchylky od trajektorie je na modelu Robot přidán objekt Graph, který umožňuje real time vykreslování odchylky. (OBR. 3-25)



Obr. 3-24 Hierarchie robota

Obr. 3-25 Graf odchylky

3.2.2 Řídicí script

Tento script obsahuje veškeré algoritmy a funkce, které slouží k řízení robota v simulaci. Jedná se o child non-threaded script v jazyce Python. Program je implementován ve funkcích, které podporuje CoppeliaSim. Jedná se o funkce `sysCall_init()`, `sysCall_actuation()` a `sysCall_sensing()`. Inicializační funkce `sysCall_init()` slouží primárně k inicializaci globálních parametrů, které se ve scriptu používají. Funkce `sysCall_actuation()` čte data ze senzorů, aktualizuje proměnné a na základě výstupní proměnné z controlleru volá jednotlivé algoritmy pro ovládání robota. Funkce `sysCall_sensing()` slouží k vykreslení reálné stopy za robotem, díky této stopě lze porovnat, jak robot sleduje trajektorii.

3.2.2.1 Controller

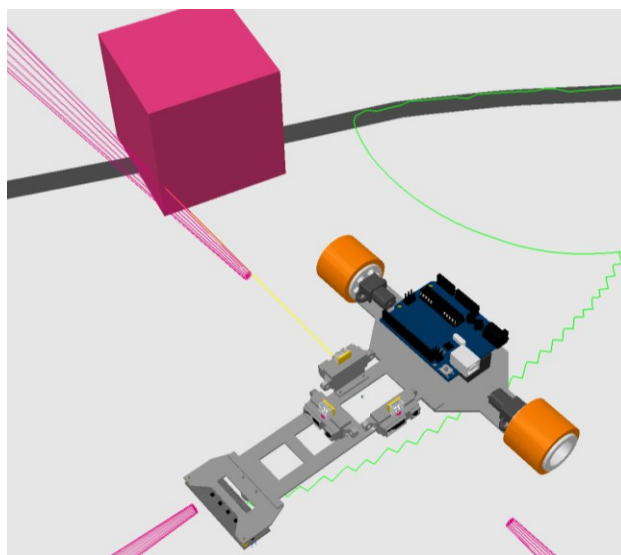
Funkce `controller()` slouží k rozhodování jaký algoritmus má robot použít. V závislosti na vstupních a globálních parametrech volí výstupní stav a zároveň aktualizuje globální parametry. Dle výstupu z této funkce jsou následně ve funkci `sysCall_actuation()` volány jednotlivé algoritmy. Controller testuje jednotlivé podmínky a následně volí nový stav. (PŘÍLOHA A)

3.2.2.2 Implementace algoritmu pro sledování trajektorie

Algoritmus pro sledování trajektorie je implementován dle předchozích úvah. (KAPITOLA 1.11) Pro zjištění odchylky a zároveň zjištění, jestli se robot nachází na trajektorii, slouží funkce `get_trajectory_deviation()`. Odchylka je vypočítána pomocí váženého průměru ze vstupu z visual sensorů. Funkce `line_follow_algorithm()` (PŘÍLOHA B) dle vstupní odchylky a nastavených parametrů pro PID controller vypočítá a aktualizuje rychlosti jednotlivých motorů robota. (OBR. 3-26)

3.2.2.3 Implementace algoritmu pro objíždění překážky

Tento algoritmus je implementovaný ve funkci `wall_follow_algorithm()`. (PŘÍLOHA D) Algoritmus při spuštění nalezne minimální vzdálenost mezi senzorem umístěným na boku robota, a objížděnou překážkou. Minimum lze detekovat při otáčení robota na místě a snímání vzdáleností od překážky. Následně se spustí objíždění překážky. Pokud je rozdíl aktuální vzdálenosti od zjištěného minima menší než tolerance, robot se pohybuje dopředu. Pokud ne robot se otáčí doprava. (OBR. 3-26)



Obr. 3-26 Ukázka simulace

3.2.2.4 Funkce hledání trajektorie

Při vyjetí z trajektorie se robot musí dostat zpět na trať. Tento problém vzniká hlavně v ostrých zatáčkách a při křižovatkách. Funkce `seeking_line()` rozhoduje o tom v jakém směru se bude robot otáčet. V závislosti na poslední průměrné odchylce se volí směr otáčení robota. V případě že robot v posledních 5 vteřinách vyjel z trajektorie, dojde k otáčení robota na místě. Pokud dojde k překonání tohoto času, robot se bude pohybovat po kružnici a v závislosti na čase se bude zvyšovat její průměr. Robot je schopný reagovat pokud vyjede z ostré zatáčky a zároveň pokud nemůže najít trajektorii zvětšuje dosah ve kterém ji je schopný detekovat. (PŘÍLOHA C)

3.2.3 Generátor trajektorie a překážek

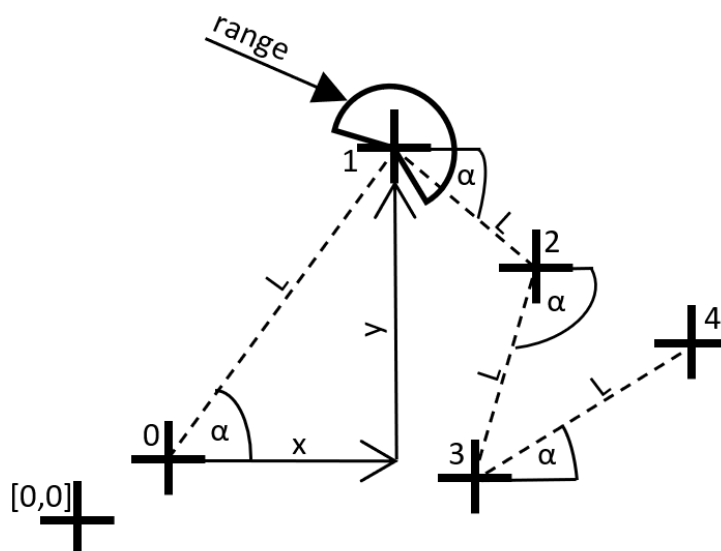
Pro vytvoření simulační smyčky slouží skript generator. Tento skript je opět non-threaded skript v jazyce Python. Hlavní účel tohoto skriptu je generace nové trajektorie a překážky, v závislosti na aktuální pozici robota. Při startu simulace dochází k vytvoření náhodně generované trajektorie. Pokud robot úspěšně sleduje trajektorii a dostane se k jejímu konci, dojde k prodloužení trajektorie a odstranění trajektorie za robotem. K prodloužení trajektorie dojde, pokud se robot nachází v definovaném kolizním prostoru na konci aktuální trajektorie.

Z důvodu zajištění simulační smyčky, je zde dále implementovaný časovač. Časovač měří dobu robota na aktuální trajektorii. Tzn. dobu od posledního prodloužení, nebo od startu simulace. Pokud dojde k překonání nastaveného času, dojde k restartu simulace. Pokud by se robot dostal z prostoru, ve kterém se má pohybovat, dojde opět k restartu simulace.

3.2.3.1 Algoritmus pro generaci trajektorie

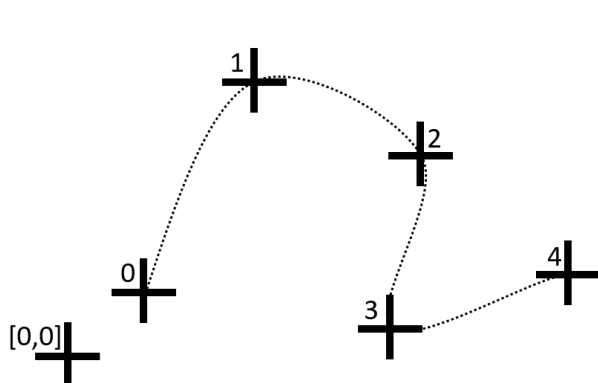
Trajektorie je vygenerována jako objekt shape z objektu typu path. CoppeliaSim sice umožňuje přímou generaci při vytvoření objektu path, ale protože je složité nastavit její rozměry je použita funkce `generateShapeFromPath()`. Tato funkce je schopná vygenerovat shape z definovaných bodů daných souřadnicemi $[x,y,z]$.

Pro vytvoření bodů slouží algoritmus `create_new_point()`. (PŘÍLOHA E) V závislosti na souřadnicích předchozího bodu a úhlu ze kterého byly souřadnice daného bodu vytvořeny, algoritmus vytvoří souřadnice nového bodu. Prostor, ve kterém se může trajektorie nacházet je omezen. Z toho důvodu jsou v algoritmu implementovány okrajové podmínky, které zamezí, aby body byly generovány uvnitř definovaného prostoru. Dále lze tento algoritmus volat za sebou a vytvořit řetězec bodů. (OBR. 3-27)

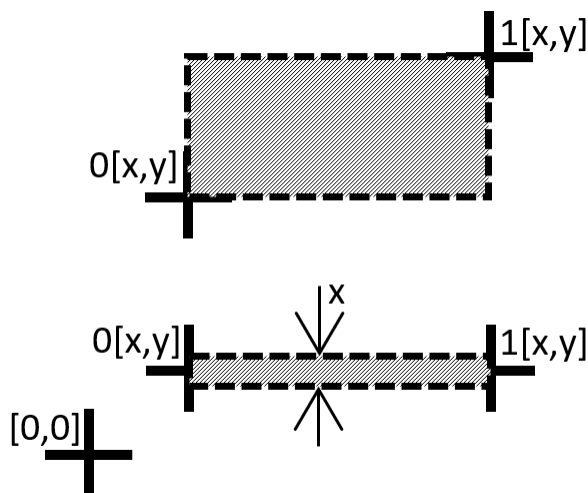


Obr. 3-27 Řetězec bodů

Z tohoto řetězce bodů je následně vytvořený objekt Path. Při vytvoření tohoto objektu dojde k dle definovaných vstupních parametrů k doplnění bodů mezi body vstupního řetězce. Tento nový list bodů je následně použit pro vytvoření finálního objektu shape, což je viditelná trajektorie v simulaci. (OBR. 3-28)



Obr. 3-28 Princip doplnění bodů



Obr. 3-29 Detekce kolize

3.2.3.2 Generace překážek

Překážka je pevný objekt, který je přesouván do středu trajektorie. Díky známým bodům z objektu path je možné překážku umístit na trajektorii. Šance na umístění této překážky na trať je možno měnit. Pokud překážka nemá být umístěna na trať nachází se v rohu simulace. K přesunu dané překážky slouží funkce `move_obstacle()` a k zjištění bodu na který se má překážka přesunout slouží funkce `get_mid_point()`.

3.2.3.3 Detekce konce trajektorie

Pokud se robot dostane ke konci trajektorie, dojde k její prodloužení. Kolizní prostor a detekce robota je realizována pomocí funkce `is_in_boundary()`. Tato funkce detekuje, jestli se třetí bod nachází uvnitř prostoru definovaném 2 body. Prostor definovaný těmito body lze vidět na OBR. 3-29. Souřadnice vstupních bodů mohou být v x nebo y souřadnici podobné a kolizní prostor by byl pro detekci malý. Z toho důvodu je v kódu implementována konstanta `x`, která prostor zvětšuje. Vstupní body jsou opět definovány z objektu path.

3.2.4 Kamera

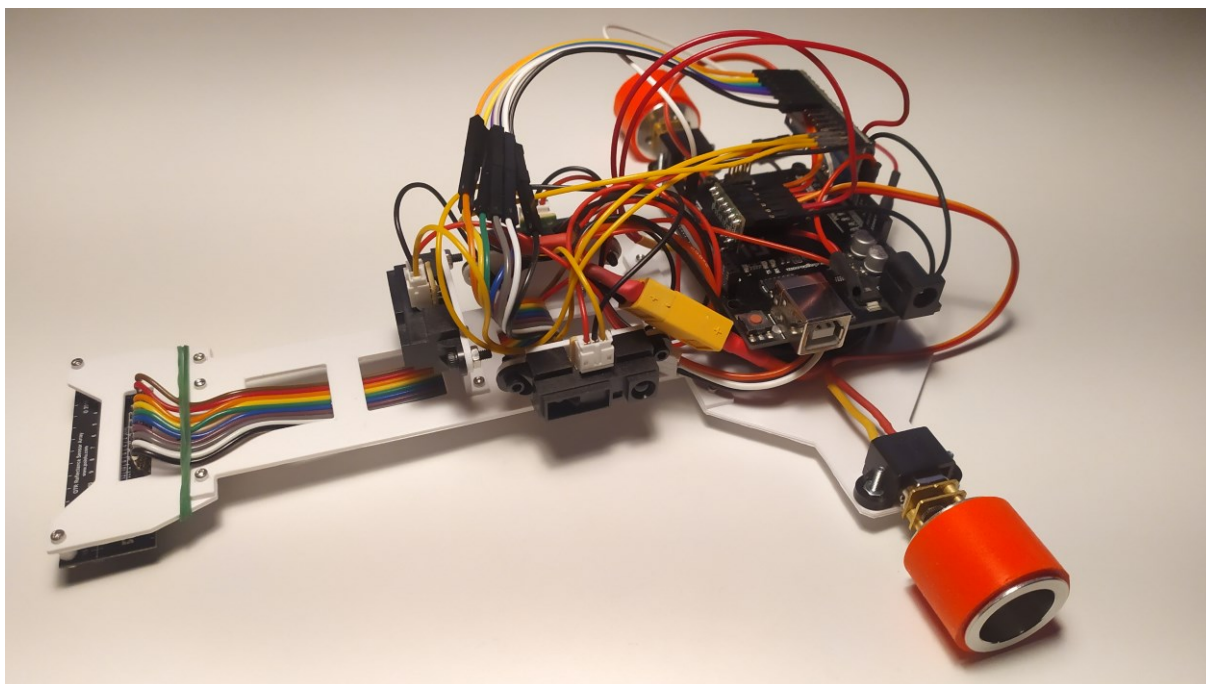
Pro lepší prezentaci simulace slouží přidaná kamera. Skript podle nastavení pozice kamery před startem simulace aktualizuje její pozici a natočení tak, aby sledovala robota.

4 Realizace

Model robota byl realizovaný dle podrobného modelu vytvořeného v Creo Parametric. Desky nosné konstrukce a L-konzoly byly vytisknuty na FDM 3d tiskárně z materiálu PLA. Pro celkové zapojení do Arduina bylo nutné připájet na driver pro motory a pro MUX piny. Samotné kabely propojující veškerou elektroniku byly barevně odlišeny. Cena komponent robota se pohybovala okolo 6000Kč.

Po sestavení robota a otestování elektroniky byl vytvořen řídící software. Software robota zrcadlí řídící script vytvořený pro robota v simulaci CoppeliaSim. Hlavní odlišnost od scriptu simulace je nutnost použití funkcí pro čtení dat z jednotlivých senzorů. Pro senzory vzdálenosti byla použita knihovna „SharpDistSensor“, která je dostupná přes GitHub. [31] Pro MUX a senzor trajektorie byly implementovány vlastní funkce pro čtení dat.

Prototyp robota je aktuálně schopen sledovat čáru a objíždět překážky. Při sledování trajektorie, zadní část robota zaostává vůči přední části. Pokud se zkrátí konstrukce robota, můžeme snížit účinky tohoto problému. Dále bude v budoucí práci výhodné vytvořit konstrukci jako celistvý výtisk z 3d tiskárny. Dále lze vytvořit vlastní PCB shield pro Arduino UNO, což by zjednodušilo zapojení komponent.



Obr. 4-1 Realizace robota

5 Závěr

Bakalářská práce se věnovala návrhem modelu robota sledujícího trajektorii s překážkami a vytvořením jeho simulace. Provedl jsem rešerši, která se zabývala aktuálním použitím line-follower robotů v komerční sféře, průzkumem nepoužívanějších simulačních softwarů, popisem běžně používaných senzorů pro detekci trajektorie a překážek, a algoritmy které slouží k vyřešení sledování trajektorie a řešení překážek.

Definoval jsem požadavkový list. Požadavkový list obsahuje parametry definované primárně ze soutěží pro Line-followery a potřebné softwarové funkce. Tyto funkce bylo nutné implementovat v řídicím systému robota a pro ovládání simulace. Následně jsem dle omezení požadavkového listu volil hardware daného robota a vytvořil model v softwaru Creo Parametric. Po vytvoření modelu jsem tento model převedl do simulačního softwaru CoppeliaSim a vytvořil simulační model robota.

V systému CoppeliaSim jsem vytvořil řídicí skript robota. Robot je při spuštění schopen najít trajektorii a sledovat ji. Pokud robot narazí na překážku je schopen překážku objet a dostat se zpět na trajektorii. Pro sledování trajektorie bylo nutné implementovat PID controller a řadu funkcí pro plynulé přecházení z jednoho algoritmu na druhý.

Dále jsem postupoval vytvořením simulačního skriptu generátor, který je schopen náhodně generovat trajektorii a překážky. Tato generace je závislá na postupu robota po dráze a na času, který robot v jednotlivém cyklu stráví. Tímto jsem zajistil nekonečnou simulační smyčku. Pro lepší prezentaci dané simulace jsem vytvořil skript, který zajišťuje sledování kamery za robotem.

Dodržené omezení a funkce z požadavkového listu:

- Model robota je v definovaných maximálních rozměrech: 220x222x60 mm
- Model robota má hmotnost: 161,49 g
- Robot je schopný detekovat a sledovat trajektorii
- Robot je schopný objíždět překážky
- Simulace umožňuje dynamickou generaci trajektorie a překážek
- Simulace je v nekonečném cyklu

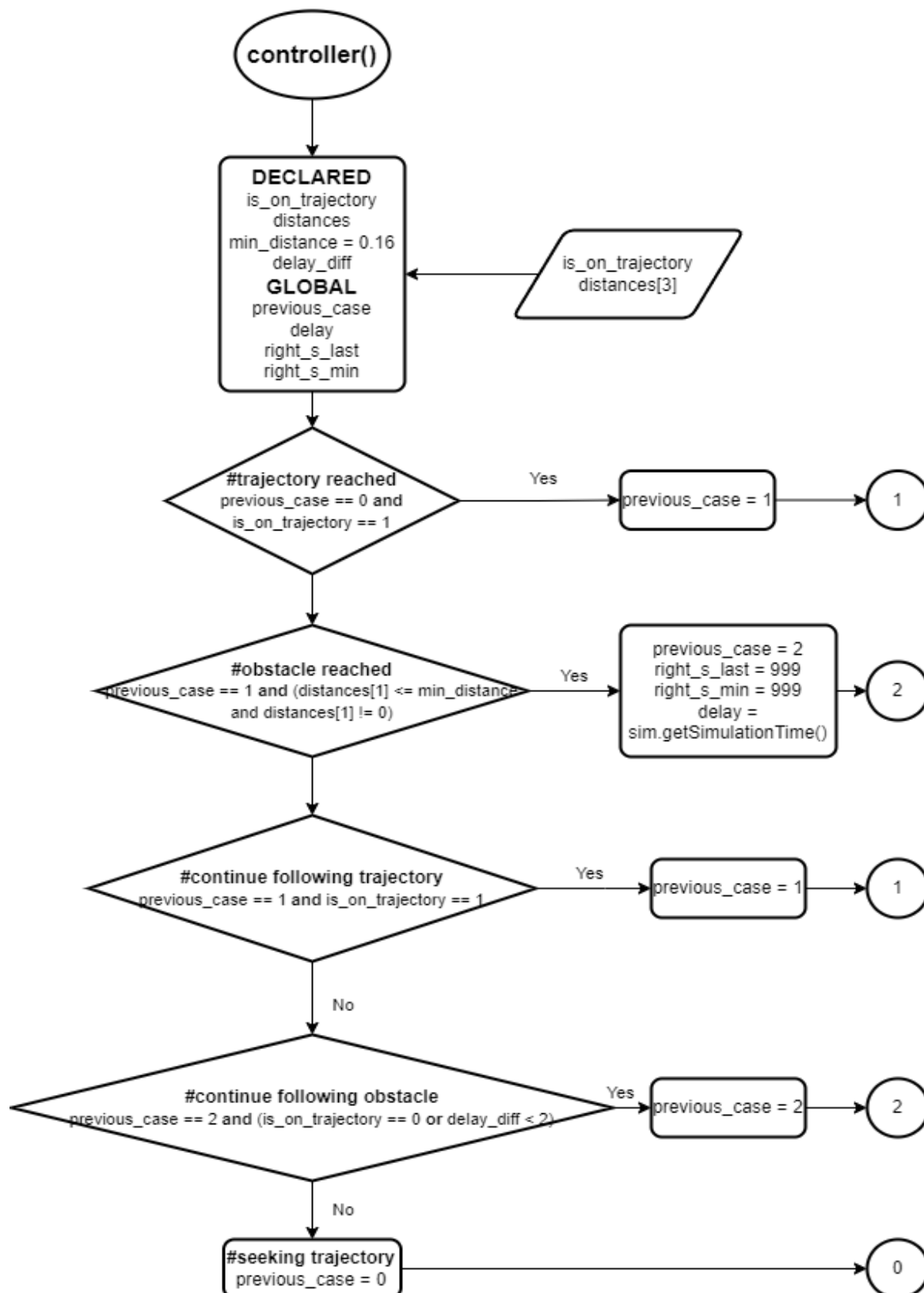
6 Poděkování

Rád bych poděkoval svému vedoucímu práce Ing. Petru Oščádalovi za čas, konzultace a zpětnou vazbu v průběhu psaní bakářské práce. Dále chci poděkovat rodině a přátelům za podporu při celou dobu mého studia.

7 Přílohy

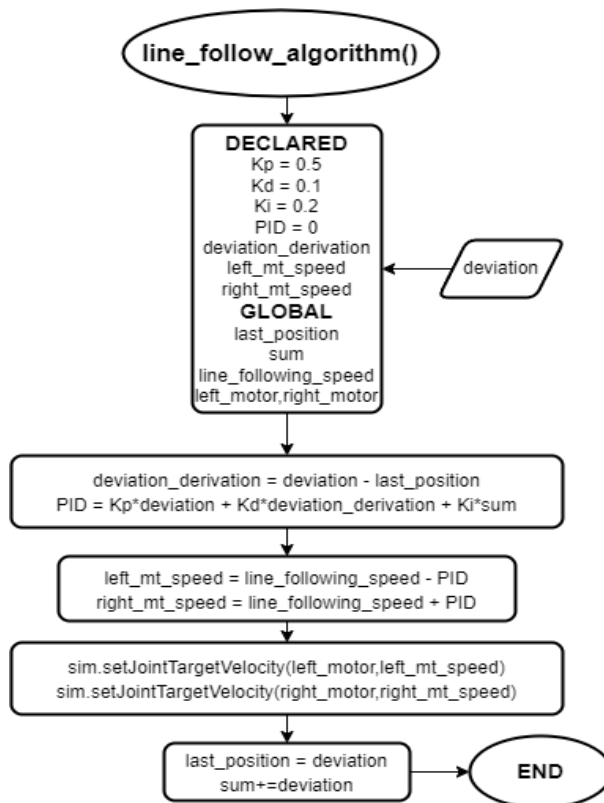
7.1 Příloha A

- diagram controller.png



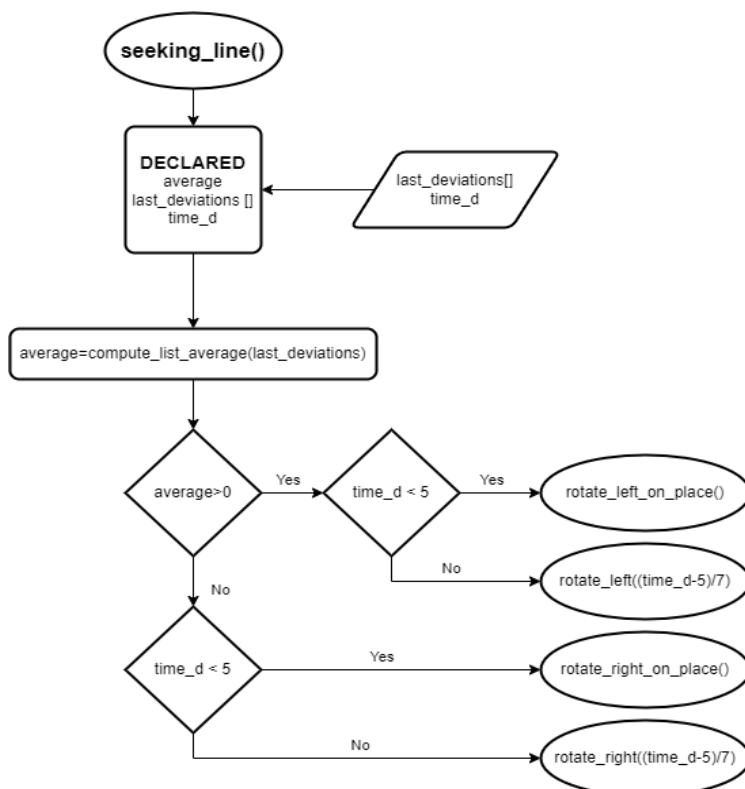
7.2 Příloha B

- diagram line_follow.png



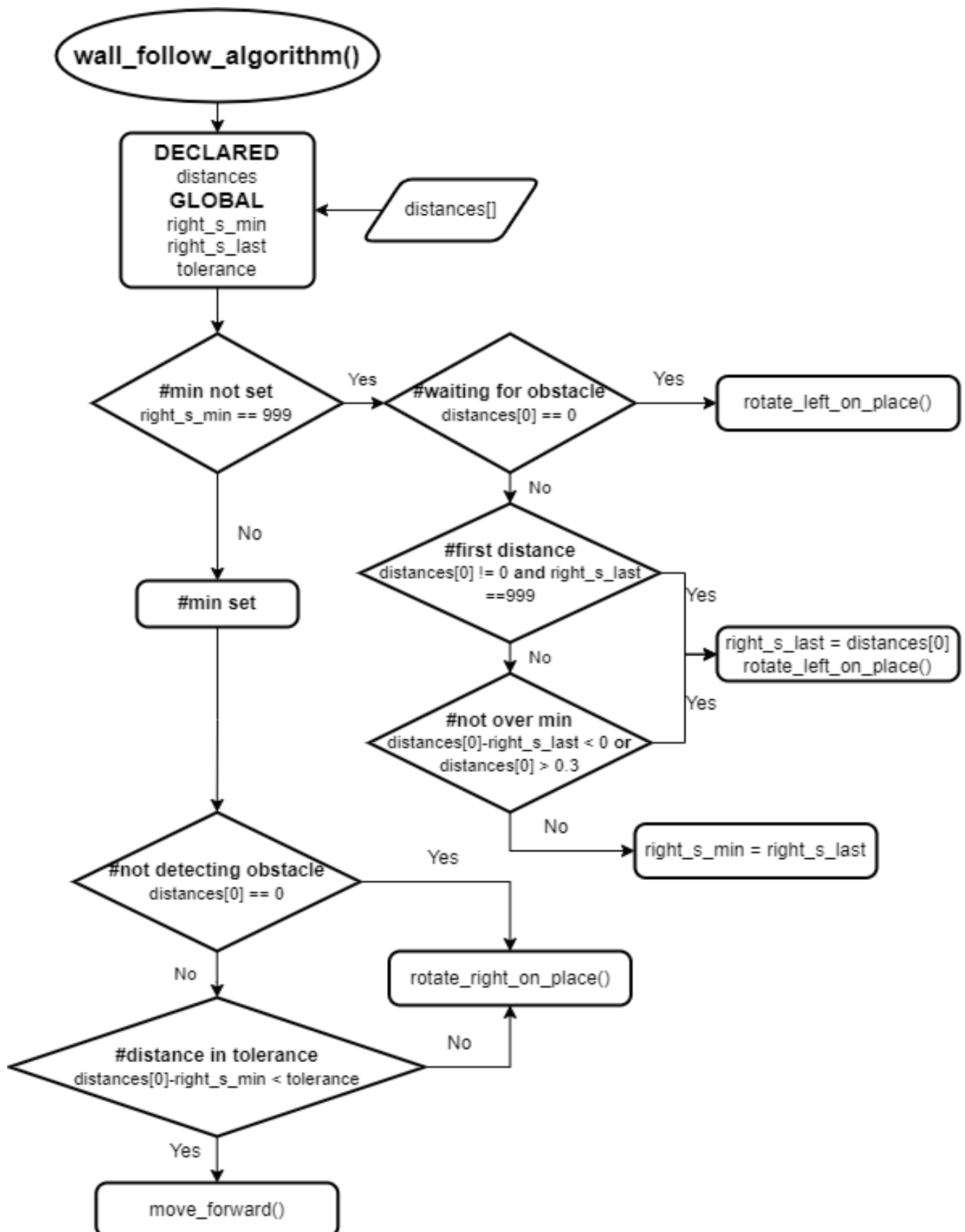
7.3 Příloha C

- diagram seeking_line.png



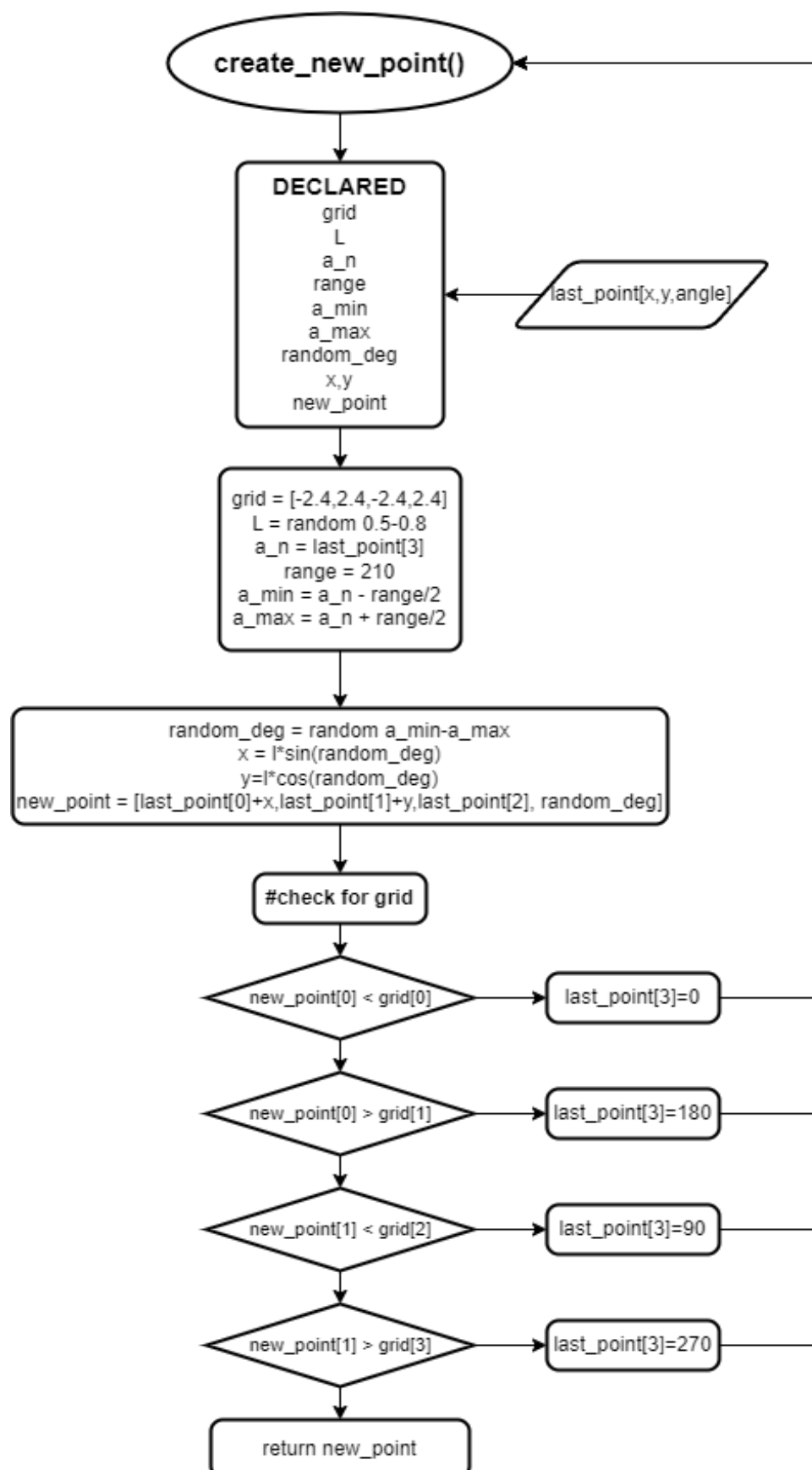
7.4 Příloha D

- diagram wall_follow.png



7.5 Příloha E

- diagram create_new_point.png



7.6 Ostatní přílohy

Příloha F: simulace.ttt - Celková simulace v CoppeliaSim

Příloha G: generator.ttt - Simulace generátoru v CoppeliaSim

Příloha H: robot_komplet.zip - Kompletní model robota

Příloha I: robot_zjednoduseny.zip – Zjednodušený model robota pro simulaci

Příloha J: arduino_code.zip – Arduino program realizace

8 Bibliografie

- [1] *History Makers* [online]. Albuquerque: David Buckley, 2022 [cit. 2022-11-02]. Dostupné z: <http://www.davidbuckley.net/DB/HistoryMakers.htm>
- [2] *1962 – “EMILY” THE WHITE-LINE FOLLOWER – BERNARD DICKMAN (AMERICAN)* [online]. Early_Mobile_Robots: cyberne1, 2011 [cit. 2022-11-02]. Dostupné z: <http://cyberneticzoo.com/early-mobile-robots/1962-emily-the-white-line-follower-bernard-dickman-american/>
- [3] LineCraft Fast Line Follower Robot Kit 4000Rpm - Assembled. In: *Sumazede* [online]. Istanbul-Turkey: Sumazede, 2020 [cit. 2022-11-02]. Dostupné z: <https://st1.myideasoft.com/idea/gg/13/myassets/products/690/2-1.jpg?revision=1632808208>
- [4] *Watch-tesla-model-s-built-robots* [online]. Washington, D.C: T.C. SOTTEK, 2013 [cit. 2022-11-03]. Dostupné z: <https://www.theverge.com/2013/7/17/4531246/watch-tesla-model-s-built-robots>
- [5] Industrial line follower robot. In: *Youtube* [online]. San Bruno, California, United States: H-TEC, 2018 [cit. 2022-11-03]. Dostupné z: https://i.ytimg.com/vi/peOM_Nk4AEY/maxresdefault.jpg
- [6] COLLINS, Jack, Shelvin CHAND, Anthony VANDERKOP a David HOWARD. A Review of Physics Simulators for Robotic Applications. *IEEE Access* [online]. USA, 2021, **9**(51416-51431), 51416-51431 [cit. 2022-11-11]. ISSN 2169-3536. Dostupné z: doi:10.1109/ACCESS.2021.3068769
- [7] Srovnání simulačních softwarů. In: *IEEE Xplore* [online]. Kanada: IEEE, 2021 [cit. 2023-05-02]. Dostupné z: <https://ieeexplore.ieee.org/document/9386154/citations?tabFilter=papers#citations>
- [8] CoppeliaSim tutorial. In: *CoppeliaRobotics* [online]. USA: CoppeliaRobotics, 2022 [cit. 2022-11-27]. Dostupné z: <https://www.coppeliarobotics.com/helpFiles/>
- [9] Magnetic sensor MGS1600C. In: *Roboteq* [online]. USA: roboteq, 2022 [cit. 2022-11-04]. Dostupné z: https://www.roboteq.com/images/2019/09/05/magsensor_numbers.png
- [10] *Roboteq* [online]. USA: roboteq, 2022 [cit. 2022-11-04]. Dostupné z: <https://www.roboteq.com/all-products/magnetic-guide-sensors>
- [11] Obrazový snímač. In: *Canon* [online]. USA: Canon, 2022 [cit. 2022-11-06]. Dostupné z: <https://canon-cmos-sensors.com/wp-content/uploads/2018/10/block-image-120mp.png>
- [12] Vision senzory. In: *CoppeliaRobotics* [online]. Švýcarsko: CoppeliaRobotics, 2022 [cit. 2023-05-02]. Dostupné z: <https://www.coppeliarobotics.com/helpFiles/en/images/visionSensors2.jpg>

-
- [13] Proximity senzory. In: *CoppeliaRobotics* [online]. Švýcarsko: CoppeliaRobotics, 2022 [cit. 2023-05-02]. Dostupné z: <https://www.coppeliarobotics.com/helpFiles/en/images/proximitySensors2.jpg>
- [14] Vážený průměr. In: *Wikipedia* [online]. None: Wikipedia org, 2022 [cit. 2022-12-16]. Dostupné z: https://en.wikipedia.org/wiki/Weighted_arithmetic_mean
- [15] SUSNEA, Ioan, Viorel MîNZU a G. VASILIU. Simple, real-time obstacle avoidance algorithm for mobile robots. In: *Https://www.researchgate.net/* [online]. Puerto de la Cruz, SPAIN: researchgate, 2009 [cit. 2022-12-17]. Dostupné z: https://www.researchgate.net/publication/228955195_Simple_real-time_obstacle_avoidance_algorithm_for_mobile_robots
- [16] Bug_1. In: *SpringerOpen* [online]. None: SpringerOpen, 2022 [cit. 2022-12-17]. Dostupné z: https://media.springernature.com/full/springer-static/image/art%3A10.1186%2Fs13638-019-1396-2/MediaObjects/13638_2019_1396_Fig1_HTML.png?as=webp
- [17] Bug_2. In: *SpringerOpen* [online]. None: SpringerOpen, 2022 [cit. 2022-12-17]. Dostupné z: https://media.springernature.com/full/springer-static/image/art%3A10.1186%2Fs13638-019-1396-2/MediaObjects/13638_2019_1396_Fig2_HTML.png?as=webp
- [18] Bubble algoritmus. In: *RobotShop* [online]. None: RobotShop, 2022 [cit. 2022-12-17]. Dostupné z: <https://community.robotshop.com/forum/uploads/default/original/2X/8/8a5f8ebb33f0880a0495cb9aa7179e4677adcc59.pdf>
- [19] Pravidla soutěží. In: *UkMars* [online]. United Kingdom: UkMars, 2022 [cit. 2023-01-08]. Dostupné z: <https://ukmars.org/contests/contest-rules/line-follower/>
- [20] Micro Metal Gearmotor. In: *Pololu* [online]. Las Vegas: Pololu, 2023 [cit. 2023-01-23]. Dostupné z: <https://www.pololu.com/product/3062/specs>
- [21] DRV8835 Driver. In: *Pololu* [online]. Las Vegas: Pololu, 2023 [cit. 2023-01-23]. Dostupné z: <https://www.pololu.com/product/2135/resources>
- [22] JS2622 kola. In: *JSumo* [online]. Turecko: JSumo, 2023 [cit. 2023-01-23]. Dostupné z: <https://www.jsumo.com/js2622-aluminum-silicone-wheel-pair>
- [23] Whell Caster. In: *Pololu* [online]. Las Vegas: Pololu, 2023 [cit. 2023-01-23]. Dostupné z: <https://www.pololu.com/product/950>
- [24] Přesnost senzoru trajektorie. In: *Solarbotics* [online]. Kanada: solarbotics, 2011 [cit. 2023-01-19]. Dostupné z: <https://www.solarbotics.com/wp-content/uploads/airrs-manual-35238-nov-7-2011.pdf>
-

-
- [25] Senzor vzdálenosti. In: *Pololu* [online]. Las Vegas: Pololu, 2023 [cit. 2023-01-23]. Dostupné z: <https://www.pololu.com/product/136/specs>
- [26] Senzor trajektorie. In: *Pololu* [online]. Las Vegas: Pololu, 2023 [cit. 2023-01-23]. Dostupné z: <https://www.pololu.com/product/4248>
- [27] Multiplexer. In: *Laskakit* [online]. Česká republika: Laskakit, 2023 [cit. 2023-01-23]. Dostupné z: <https://www.laskakit.cz/analogovy-multiplexer-16-kanalu-cd74hc4067/#relatedFiles>
- [28] Arduino Uno. In: *Arduino Store* [online]. Austrálie: Arduino Store, 2023 [cit. 2023-01-23]. Dostupné z: <https://store.arduino.cc/products/arduino-uno-rev3>
- [29] Baterie G3 Ray. In: *ProfiModel* [online]. Česká republika: ProfiModel, 2023 [cit. 2023-01-23]. Dostupné z: <https://profimodel.cz/cs/do-1000mah/38331-g3-ray-li-pol-450mah-74-30-60c-air-pack-33wh-8595241350738.html>
- [30] CoppeliaSim helpfiles. In: *CoppeliaRobotics* [online]. Švýcarsko: CoppeliaRobotics, 2022 [cit. 2023-03-07]. Dostupné z: <https://www.coppeliarobotics.com/helpFiles/>
- [31] SharpDistSensor. In: *Github* [online]. USA: DrGFreeman, 2020 [cit. 2023-05-10]. Dostupné z: <https://github.com/DrGFreeman/SharpDistSensor>